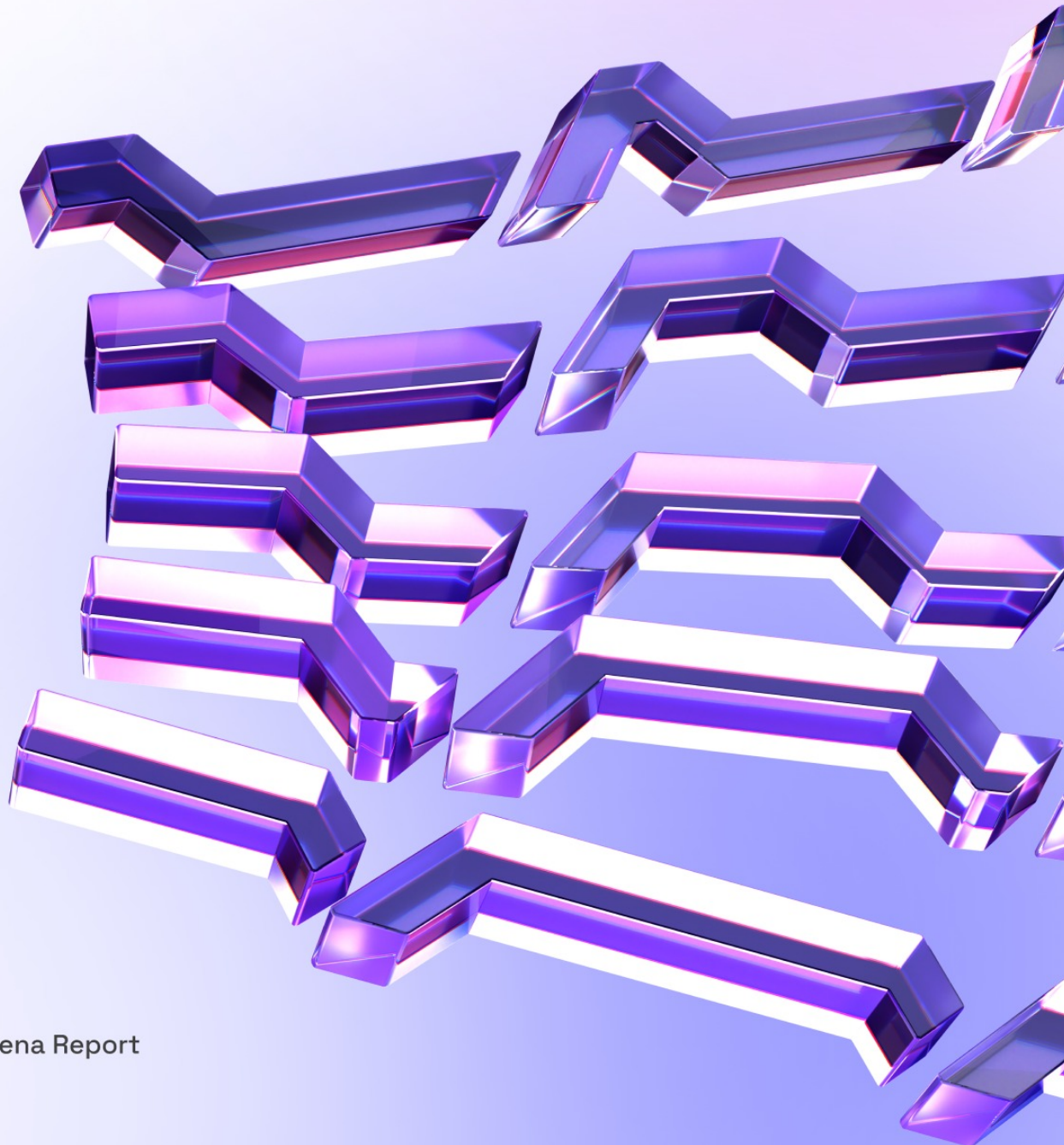




IPOR-Labs

ipor-fusion

December 9, 2025 13:19 UTC



Contents

- 1. Document Revisions 3
- 2. Overview 4
 - 2.1. Ackee Blockchain Security 4
 - 2.2. Wake Arena 5
 - 2.3. Disclaimer 5
 - 2.4. Finding Classification 6
- 3. Executive Summary 7
 - Revision 1.0 7
- 4. Findings Summary 11
- Report Revision 1.0 19
 - Findings 19
- Appendix A: How to cite 170

1. Document Revisions

1.0	AI Scan	Dec 9, 2025 13:19 UTC
---------------------	---------	-----------------------

2. Overview

This document presents findings identified by [Wake Arena](#) automated analysis.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling [Wake](#) for Ethereum and [Trident](#) for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the [School of Solana](#) and the [Solana Auditors Bootcamp](#).

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.

Rohanske nabrezi 717/4

186 00 Prague, Czech Republic

<https://ackee.xyz>

hello@ackee.xyz

2.2. Wake Arena

This report was generated using [Wake Arena](#), an automated vulnerability analysis tool. Wake Arena utilizes [Wake](#) with additional detectors to perform comprehensive AI and static analysis.

To identify potential vulnerabilities and issues in smart contracts [Wake](#) framework utilizes:

- Code structure and patterns
- Control flow graph
- Data flow graph
- Common vulnerability patterns
- Contract interactions

The findings presented in this report are based on automated analysis optimized for precision, aiming for a low false-positive rate. The detection is not optimized for recall—it doesn't target finding all issues (which come at the cost of a high false-positive rate). This code review should be complemented with additional manual code review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, however automated findings shouldn't be considered as a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

IPOR-Labs/ipor-fusion was audited at commit

[1e25bb68c4f3b33fd0b6ce20f17f70e2d5b18b40](#), focusing on the Plasma Vault (ERC-4626 upgradeable vault), access-management libraries, oracle middleware, reward and withdraw managers, pre-hooks, math utilities, a universal token swapper, and associated libraries. The review covered the contracts enumerated in the scope and their interactions across vault accounting, fee accrual, access control, oracles, and swap execution.

A total of 41 AI-detected issues were identified across the codebase. The distribution shows concentrated risk in authorization flows, accounting/units, ERC-4626 semantics, and external call surfaces. The most severe detected issues include: critical unit mismatch in

`PlasmaVaultMarketsLib.updateMarketsBalances` that undercounts market assets by `10^DECIMALS_OFFSET` and propagates into `_getGrossTotalAssets` and limit checks; and a critical post-execution sweep in `SwapExecutorEth.execute` that transfers all balances to `msg.sender`, enabling mempool copy theft and public draining of stray ETH/ERC20 held by the contract.

High-impact findings further illuminate systemic weaknesses: missing minimum-shares slippage control in `ERC4626ZapIn.zapIn` allows front-running to dilute minted shares; the Chainlink fallback in

`PriceOracleMiddlewareWithRoles._getAssetPrice` lacks staleness and round-completeness validation, risking stale or incomplete prices;

`RedemptionDelayLib` enables third-party lock refreshes on a victim via receiver-directed deposits and exposes a public `canCallAndUpdate` path that can refresh locks without a real deposit; `RewardsClaimManager.claimRewards` forwards arbitrary delegatecalls into supported fuses, allowing privilege

escalation from the claim role; and `SwapExecutorEth.execute` permits arbitrary external calls that can seed persistent ERC20 allowances from the contract to attacker-controlled spenders.

Overall, while the architecture is modular and leverages OpenZeppelin upgradeable components, the security posture is currently weak for production use. Authorization boundaries blur under delegatecall-based flows, accounting and unit conversions contain high-severity logic errors, ERC-4626 invariants are not consistently respected, and several public entry points allow unsafe arbitrary execution and value sweeping. Until these issues are addressed with strict recipient binding and signatures, stronger oracle validation, corrected unit math, ERC-4626-conformant previews and transfers, and tighter call-selector whitelisting, the codebase is not ready for deployment to adversarial environments.

Key Technical Findings

- Critical: `PlasmaVaultMarketsLib.updateMarketsBalances` stores market balances using `(decimals - DECIMALS_OFFSET)` then consumes them as full decimals, undercounting markets by $10^{\text{DECIMALS_OFFSET}}$ and corrupting `_getGrossTotalAssets` and limit checks.
- Critical: `SwapExecutorEth.execute` sweeps `tokenIn`, `tokenOut`, WETH for ETH, and arbitrary `tokensDustToCheck` to `msg.sender`, enabling mempool copy theft and public dust draining; outcomes are not bound to an authenticated recipient.
- High: `ERC4626ZapIn.zapIn` lacks a minimum-shares slippage check and does not use `previewDeposit/convertToShares`, allowing front-running to reduce shares minted.
- High: `PriceOracleMiddlewareWithRoles._getAssetPrice` accepts Chainlink registry answers without freshness and round-completeness checks, permitting stale or incomplete USD prices to propagate.

- High: `RedemptionDelayLib.lockChecks` refreshes a receiver's redemption lock on deposits/mints; ERC-4626 allows depositing to arbitrary `receiver`, enabling griefing via dust deposits that keep a victim locked.
- High: Public `canCallAndUpdate` combined with `RedemptionDelayLib.lockChecks` lets anyone refresh a victim's lock by calling with a deposit selector and arbitrary `account_`, without a real deposit.
- High: `RewardsClaimManager.claimRewards` forwards arbitrary `FuseAction.data` for delegatecall in the vault context, enabling privilege escalation from `CLAIM_REWARDS_ROLE` to broader market operations.
- High: `SwapExecutorEth.execute` arbitrary external calls allow seeding persistent ERC20 approvals from this contract to attacker-controlled spenders; approvals persist and can later exfiltrate funds.
- Medium: ERC-4626 semantics deviations in `PlasmaVault.withdraw` subtracts fees from assets (returning less than requested) and `previewWithdraw` divides by fee instead of applying $1/(1 - \text{fee})$, inflating required shares.
- Medium: `PlasmaVault._getGrossTotalAssets` counts rewards held in `RewardsClaimManager`, inflating NAV and affecting previews/limits.
- Medium: `ContextManager.runWithContextAndSignature` uses a monotonic high-nonce acceptance that can invalidate lower nonces, enabling grief/DoS for intended sequential policies.
- Medium: `IporMath` rounding and edge-case overflows in `divisionInt`, `convertWadToAssetDecimals`, and `convertToWadInt` risk unexpected reverts or bias under extreme decimals.
- Warning: Authorization special-casing by selector in `AccessManagedUpgradeable._checkCanCall` risks drift and misclassification on collisions; supply cap units mismatch in `PlasmaVaultBase.cap` (shares vs assets); Y2106 downcasts in fee timestamp paths.

Conclusion

The presence of 2 critical and 8 high AI-detected issues across accounting, authorization, oracle validation, and swap execution indicates the codebase is not ready for deployment. Immediate remediation of unit mismatches, unsafe sweeping and arbitrary-call surfaces, ERC-4626 semantic breaks, redemption lock abuses, and oracle validation gaps is required before mainnet exposure. After fixes, add explicit recipient binding with EIP-712 intents, strict selector whitelisting for delegatecalls, comprehensive Chainlink validations, and corrected fee and preview math, followed by re-audit and scenario testing.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
2	8	12	6	7	6	41

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
C1: Post-execution sweep sends all balances to msg.sender, enabling mempool copy theft and public dust draining	Critical	1.0	Reported
C2: Unit mismatch in updateMarketsBalances undercounts market assets by 10^DECIMALS_OFFSET, corrupting total assets, fees, and risk limits	Critical	1.0	Reported
H1: Missing min-shares slippage check on ERC4626 deposit allows front-running to reduce shares minted	High	1.0	Reported
H2: Chainlink fallback lacks staleness and round-completeness checks, allowing stale or incomplete USD prices	High	1.0	Reported

Finding title	Impact	Reported	Status
H3: Receiver-directed deposits allow third parties to refresh a victim's redemption lock (DoS)	High	1.0	Reported
H4: Iteration over hooks stops at first empty slot, skipping valid hooks after gaps	High	1.0	Reported
H5: Performance fee shares over-minted due to missing normalization by current exchange rate	High	1.0	Reported
H6: claimRewards allows arbitrary fuse delegatecalls, enabling privilege escalation from CLAIM_REWARDS_ROLE	High	1.0	Reported
H7: Public canCallAndUpdate lets anyone set a victim's redemption lock without a real deposit	High	1.0	Reported
H8: Arbitrary external calls allow persistent ERC20 approvals from this contract to attacker-controlled spenders	High	1.0	Reported

Finding title	Impact	Reported	Status
M1: withdraw transfers fewer assets than requested by subtracting fee from assets, violating ERC4626 withdraw semantics	Medium	1.0	Reported
M2: divisionInt negates int256.min when taking absolute value, causing overflow and revert	Medium	1.0	Reported
M3: Unrestricted arbitrary external calls enable theft of contract-held tokens and persistent approvals	Medium	1.0	Reported
M4: convertWadToAssetDecimals overflows on scaling 10^{**}delta or on multiplication for extreme decimals	Medium	1.0	Reported
M5: convertToWadInt cast of 10^{**}delta to int256 can revert for extreme decimals	Medium	1.0	Reported
M6: previewWithdraw divides by fee instead of applying discount $1/(1 - \text{fee})$, inflating required shares and breaking ERC4626 previews	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M7: User-controlled sweeping of arbitrary token balances enables cross-user theft of stray funds	Medium	1.0	Reported
M8: Monotonic nonce check allows high-nonce execution to invalidate lower nonces (violates sequential policy), enabling DoS/griefing	Medium	1.0	Reported
M9: divisionInt rounds negative half ties toward zero, violating nearest rounding	Medium	1.0	Reported
M10: Redemption lock is global per account across a shared Access Manager, enabling cross-vault DoS	Medium	1.0	Reported
M11: totalAssets counts rewards held in RewardsClaimManager, inflating NAV and breaking convert/preview/limits	Medium	1.0	Reported
M12: Unrestricted call loop can forward contract-held ETH and leverage existing approvals	Medium	1.0	Reported

Finding title	Impact	Reported	Status
L1: Stale or zero lastValidatedTimestamp returned: mapping is never updated nor cleared	Low	1.0	Reported
L2: division docstring claims nearest rounding but implementation truncates toward zero	Low	1.0	Reported
L3: Unconditional substrates write persists entries under address(0) during removal and no-op updates	Low	1.0	Reported
L4: Integer division in 1% tolerance check permits up to ~1.99% deviation	Low	1.0	Reported
L5: clearContext does not detect missing context due to getSenderFromContext fallback, allowing no-op clears and a misleading ContextCleared event	Low	1.0	Reported
L6: Stale mapping entries in configureInstantWithdrawal Fuses cause permanent storage bloat when reducing or reordering the fuse list	Low	1.0	Reported

Finding title	Impact	Reported	Status
W1: Supply cap unit mismatch: documentation claims underlying units while enforcement uses shares	Warning	1.0	Reported
W2: Direct call to <code>getCallbackHandlers(address[], bytes4[])</code> reads wrong storage context and returns incorrect data	Warning	1.0	Reported
W3: Direct call to <code>getCallbackHandler(address, bytes4)</code> reads wrong storage context and returns incorrect handler	Warning	1.0	Reported
W4: Timestamp downcast to <code>uint32</code> in <code>updateManagementFeeData</code> will revert after 2¹⁰⁶, bricking fee-related code paths	Warning	1.0	Reported
W5: Documentation mismatch: <code>canWithdrawFromUnallocated</code> reserves only released shares, not all pending requests	Warning	1.0	Reported

Finding title	Impact	Reported	Status
W6: Hardcoded selector special-casing in <u>_checkCanCall</u> risks authorization drift and selector-collision misclassification	Warning	1.0	Reported
W7: Global redemption lock scoped per AccessManager (not per vault) enables cross-vault lockouts	Warning	1.0	Reported
I1: Y2106 timestamp downcast in fee timestamp update can revert and halt management fee realization (triggered by <u>prepareForRealizeManagement Fee</u>)	Info	1.0	Reported
I2: Negative price answers revert via SafeCast before domain-specific checks, causing inconsistent errors	Info	1.0	Reported
I3: Misleading NatSpec: function documented as public/no restrictions but guarded by AccessManager <u>restricted</u> modifier	Info	1.0	Reported

Finding title	Impact	Reported	Status
I4: Misleading access control: NatSpec says owner-only, but function is role-gated (SET_ASSETS PRICES SOURCES)	Info	1.0	Reported
I5: Unused struct CallbackHandlerInfo increases code surface without benefit	Info	1.0	Reported
I6: NatSpec promises decimals-mismatch reverts, but implementation never validates decimals	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision.
For the complete list of all findings, [Go back to Findings Summary](#)

C1: Post-execution sweep sends all balances to msg.sender, enabling mempool copy theft and public dust draining

Critical impact finding

Impact:	Critical	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorEth.sol	Type:	Front-running

Description

The function `execute` unconditionally sweeps any balances held by this contract to `msg.sender` at the end of execution. There is no binding to an explicit `recipient`, no signature-based authorization, and the caller controls all pre-sweep external calls. As a result, any third party can redirect a victim's intended proceeds by copying their calldata and submitting it first, and anyone can drain stray ERC20 or ETH left on the contract by invoking `execute` with those assets listed.

This behavior is unsafe in adversarial, public mempool environments where transactions are observable and replayable prior to inclusion.

Listing 1. Code snippet from

contracts/fuses/universal_token_swapper/SwapExecutorEth.sol

```
85     if (balanceTokenIn > 0) {
86         IERC20(data_.tokenIn).safeTransfer(msg.sender, balanceTokenIn);
87     }
88
89     if (balanceTokenOut > 0) {
90         IERC20(data_.tokenOut).safeTransfer(msg.sender, balanceTokenOut);
91     }
92
93     uint256 balanceEth = address(this).balance;
```

```

94
95     if (balanceEth > 0) {
96         IWETH9(W_ETH).deposit{value: balanceEth}();
97         IERC20(W_ETH).safeTransfer(msg.sender, balanceEth);
98     }
99
100     uint256 lenDustToCheck = data_.tokensDustToCheck.length;
101     uint256 dustBalance;
102     for (uint256 i; i < lenDustToCheck; ++i) {
103         dustBalance =
104         IERC20(data_.tokensDustToCheck[i]).balanceOf(address(this));
105         if (dustBalance > 0) {
106             IERC20(data_.tokensDustToCheck[i]).safeTransfer(msg.sender,
107                 dustBalance);
108         }
109     }

```

The snippet shows that any residual `tokenIn`, `tokenOut`, and each `tokensDustToCheck` balance is transferred to `msg.sender`. Any ETH balance is converted to WETH and transferred to `msg.sender` as well. None of these transfers are tied to an intended recipient or restricted by policy.

Exploit Scenario

A realistic attack leverages observable calldata and the hard-coded `msg.sender` recipient to steal swap outputs or drain dust.

Consider the following attack scenario:

1. Alice prepares a transaction with `execute`, including an ERC-2612 `permit` and a DEX swap in `callDatas` that will move her `tokenIn` into the executor and swap to `tokenOut`.
2. Bob, watching the mempool, copies Alice's exact calldata and submits it with a higher fee.
3. The contract executes the calls using Bob's transaction; Alice's `permit` is consumed, her funds are pulled and swapped, and the post-execution sweep sends all balances to Bob's `msg.sender`.

4. Alice's original transaction then fails or executes with nothing left to sweep. Bob keeps the entire `tokenOut` amount.

A second scenario drains stray balances:

1. Alice notices that the executor holds ETH or an ERC20 (e.g., accidental transfers, refunds, or prior dust).
2. Alice calls `execute` with empty `targets/callDatas`, setting `tokenIn`, `tokenOut`, or `tokensDustToCheck` to include those assets.
3. The function sweeps the contract's balances of those assets to Alice's `msg.sender`.

Recommendation

Bind the outcome to an authenticated recipient and remove public sweeping of arbitrary balances.

Fix the vulnerability by:

- Adding an explicit `recipient` field to the input and transferring all proceeds to `recipient`, not `msg.sender`.
- Requiring an EIP-712 signature over the full intent, including `recipient`, `targets`, `callDatas`, `ethAmounts`, `tokenIn`, `tokenOut`, `tokensDustToCheck`, `chainId`, `nonce`, and `deadline`. Verify the signature on-chain and enforce replay protection by tracking nonces.
- Restricting sweeping to known outputs only. Do not accept an arbitrary `tokensDustToCheck` list from the caller; instead, send only `tokenOut` (and optionally unspent `tokenIn`) to `recipient`. Provide an admin-only `rescueTokens` function for accidental transfers.
- Optionally restricting callable `targets`/selectors to vetted routers to reduce abuse surface.
- Emitting to `recipient` and validating that `recipient` is the authorized signer

or an approved relayer.

As a defense-in-depth measure, consider using a non-reentrancy guard around the sweeping phase and maintaining strict checks-effects-interactions ordering.

[Go back to Findings Summary](#)

C2: Unit mismatch in updateMarketsBalances undercounts market assets by $10^{\text{DECIMALS_OFFSET}}$, corrupting total assets, fees, and risk limits

Critical impact finding

Impact:	Critical	Confidence:	High
Target:	./contracts/vaults/lib/PlasmaVaultMarketsLib.sol	Type:	Logic error

Description

The function `updateMarketsBalances` converts WAD USD values into the underlying asset quantity using `IporMath.convertWadToAssetDecimals` with `(decimals_ - decimalsOffset_)`. As a result, per-market balances are persisted in storage with fewer decimals than the underlying token. These values are then consumed as if they were in full underlying decimals when computing gross total assets and evaluating market limits, causing systematic undercounting by a factor of $10^{\text{DECIMALS_OFFSET}}$.

This mismatch directly affects `_getGrossTotalAssets` by summing a full-decimals wallet balance with under-decimals market totals, and it affects `AssetDistributionProtectionLib.checkLimits` by comparing a full-decimals denominator (`totalBalanceInVault`) with an under-decimals numerator (`balanceInMarket`). The consequence is skewed accounting, undercharged fees, and weakened market limit enforcement.

Listing 2. Code snippet from

contracts/vaults/lib/PlasmaVaultMarketsLib.sol

```
48         dataToCheck.marketsToCheck[i].balanceInMarket =  
            IporMath.convertWadToAssetDecimals(  
49             IporMath.division(  
                dataToCheck.marketsToCheck[i].balanceInMarket,  
                10 ** decimalsOffset_
```



```

50          wadBalanceAmountInUSD * IporMath.BASIS_OF_POWER **
    underlyingAssePriceDecimals, underlyingAssetPrice
51      ),
52      (decimals_ - decimalsOffset_)
53  );
54
55      deltasInUnderlying = deltasInUnderlying
56
57      + PlasmaVaultLib.updateTotalAssetsInMarket(markets[i],
    dataToCheck.marketsToCheck[i].balanceInMarket);

```

The above writes `balanceInMarket` using `(decimals_ - decimalsOffset_)` and immediately persists it via `PlasmaVaultLib.updateTotalAssetsInMarket` as the market total, which other components treat as being in the underlying token decimals.

Listing 3. Code snippet from contracts/vaults/PlasmaVault.sol

```

1376     function _getGrossTotalAssets() internal view returns (uint256) {
1377         address rewardsClaimManagerAddress =
            PlasmaVaultLib.getRewardsClaimManagerAddress();
1378
1379         if (rewardsClaimManagerAddress != address(0)) {
1380             return IERC20(asset()).balanceOf(address(this)) +
                PlasmaVaultLib.getTotalAssetsInAllMarkets()
1381
1382             +
                IRewardsClaimManager(rewardsClaimManagerAddress).balanceOf();
1383         }
1384
1385         return IERC20(asset()).balanceOf(address(this)) +
            PlasmaVaultLib.getTotalAssetsInAllMarkets();
1386     }

```

`IERC20(asset()).balanceOf(address(this))` is in full underlying decimals, while `PlasmaVaultLib.getTotalAssetsInAllMarkets()` is the sum of under-decimals market totals. No scaling correction is applied here, so the market component is understated by `10DECIMALS_OFFSET`.

For completeness, the offset used system-wide is defined as follows.

Listing 4. Code snippet from contracts/libraries/PlasmaVaultLib.sol

```
50    /// @dev The offset for the underlying asset decimals in the Plasma Vault
51    uint8 public constant DECIMALS_OFFSET = 2;
```

Technical impact in production:

- Gross total assets are persistently undervalued by the market component scale factor, biasing `totalAssets`, AUM-based fee calculations, and monitoring.
- Market limit checks use a larger denominator (wallet + undercounted markets) against an undercounted per-market numerator, making limits easier to pass than configured and allowing over-concentration risk.
- Any off-chain or on-chain logic that trusts `getTotalAssetsInAllMarkets` or `_getGrossTotalAssets` for risk, fees, or accounting inherits this error.

Exploit Scenario

A realistic failure mode is limit bypass due to mismatched units during balance updates.

Consider the following attack scenario:

1. Alice, who controls authorized operations, prepares an allocation that moves most wallet assets into a single market with a configured exposure cap;
2. Alice triggers an operation that updates balances; `updateMarketsBalances` writes `balanceInMarket` using `(decimals_ - decimalsOffset_)`, and `_getGrossTotalAssets` computes `totalBalanceInVault` using full underlying decimals;
3. `AssetDistributionProtectionLib.checkLimits` compares the under-decimals `balanceInMarket` against a limit computed from the full-decimals `totalBalanceInVault`, so the ratio appears smaller than it really is and the

check passes;

4. Bob, a passive depositor, is now over-exposed to that single market beyond the configured cap. If that market suffers adverse price moves or a protocol incident, Bob incurs larger-than-intended losses.

Additional consequences include chronic undercharging of management/performance fees due to understated gross assets. This creates value transfer from fee recipients and breaks economic assumptions for integrators relying on correct AUM.

Recommendation

Align decimals across the entire accounting pipeline so that per-market totals are stored and used in full underlying token decimals.

Fix the vulnerability by either:

- In `PlasmaVaultMarketsLib.updateMarketsBalances`, convert WAD to the underlying asset using the true underlying decimals: pass `decimals_` (not `decimals_ - decimalsOffset_`) to `IporMath.convertWadToAssetDecimals`;
- Or, if storage must remain in offset decimals, consistently rescale when reading/using totals by multiplying by `10^decimalsOffset_` before summation or limit checks.

Recommended code-level change:

- Replace `(decimals_ - decimalsOffset_)` with `decimals_` in the conversion that computes `balanceInMarket` before calling `PlasmaVaultLib.updateTotalAssetsInMarket`.

Hardening and validation:

- Add an invariant test asserting: `IERC20(asset()).balanceOf(this) + sum(markets) == PlasmaVaultLib.getTotalAssetsInAllMarkets() +`

`walletBalance` after updates, within tolerance of rewards handling.

- Add unit tests for asset distribution checks verifying that a configured market cap fails when the true (full-decimals) market exposure exceeds the limit.
- Document that `getTotalAssetsInAllMarkets` and per-market totals are represented in underlying token decimals and add runtime sanity checks or events if decimals mismatch is detected.

[Go back to Findings Summary](#)

H1: Missing min-shares slippage check on ERC4626 deposit allows front-running to reduce shares minted

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/zaps/ERC4626 ZapIn.sol	Type:	Front-running

Description

The `zapIn` function deposits the entire `depositAssetBalance` into the target ERC4626 vault but never enforces a minimum number of shares to be minted. The only user-specified bound is `minAmountToDeposit`, which checks the contract's balance of the underlying `asset` before calling `vault.deposit`, but it does not constrain the number of shares minted by the vault.

In ERC4626, the price per share can be manipulated between transaction submission and inclusion (for example, via asset donations to the vault). Because `zapIn` neither uses `previewDeposit/convertToShares` nor validates the return value of `deposit`, a victim can be front-run into receiving significantly fewer shares than expected.

Listing 5. Code snippet from contracts/zaps/ERC4626ZapIn.sol

```
97     IERC4626 vault = IERC4626(zapInData_.vault);
98     uint256 depositAssetBalance =
    IERC4626(vault.asset()).balanceOf(address(this));
99
100     if (depositAssetBalance < zapInData_.minAmountToDeposit) {
101         revert InsufficientDepositAssetBalance();
102     }
103
104     vault.deposit(depositAssetBalance, zapInData_.receiver);
```

Key observations:

- `depositAssetBalance` is compared only against `minAmountToDeposit`.
- `vault.deposit(depositAssetBalance, receiver)` returns the number of shares minted per ERC4626, but the return value is ignored.
- No `minSharesOut` parameter exists and there is no share-based slippage protection.

Exploit Scenario

A realistic attack uses price-per-share manipulation to worsen the victim's execution.

Consider the following attack scenario:

1. Alice submits a `zapIn` to deposit 1,000 units of the vault `asset`, setting `minAmountToDeposit = 1,000` because she expects roughly 1,000 shares.
2. Bob observes the transaction in the mempool and donates a large amount of `asset` directly to the vault, increasing assets-per-share just before inclusion.
3. Alice's transaction executes. The `depositAssetBalance` check passes because she still holds 1,000 `asset` in the zap contract.
4. The deposit mints substantially fewer shares due to the manipulated price per share.
5. The transaction does not revert because there is no `minSharesOut` enforcement; Alice receives fewer shares than intended.

Recommendation

Enforce a minimum number of shares minted for each deposit.

Fix the vulnerability by either:

- Extending `ZapInData` with a `minSharesOut` field, capturing the return value of `vault.deposit`, and reverting unless `mintedShares >= minSharesOut`.
- Computing `expectedShares = vault.previewDeposit(depositAssetBalance)` immediately before the deposit and requiring `mintedShares >= minSharesOut`, where `minSharesOut` is provided by the user with their acceptable slippage.

Additional hardening:

- Prefer using `previewDeposit` over `convertToShares` to account for potential fees in the vault implementation.
- If deposits can be fee-on-transfer or otherwise dynamic, consider allowing `minAssetsIn` and `minSharesOut` simultaneously, and validate both.

[Go back to Findings Summary](#)

H2: Chainlink fallback lacks staleness and round-completeness checks, allowing stale or incomplete USD prices

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol	Type:	Data validation

Description

In private function `_getAssetPrice` the Chainlink Feed Registry fallback path accepts whatever the registry returns without validating data freshness or round completeness.

The implementation ignores `roundIdChainlink`, `startedAtChainlink`, `timeChainlink` and `answeredInRoundChainlink` and only uses the `chainlinkPrice`. As a result, stale values (for example when an aggregator is paused or lagging) or incomplete rounds (where `answeredInRound` is lower than `roundId`) are treated as valid. This contradicts Chainlink's recommended validations and creates a risk of system-wide mispricing for any integrator relying on this middleware.

Affected elements:

- Function: `_getAssetPrice`
- State variables: `CHAINLINK_FEED_REGISTRY`, `QUOTE_CURRENCY`, `QUOTE_CURRENCY_DECIMALS`

Listing 6. Code snippet from

contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol

```
218         try
    FeedRegistryInterface(CHAINLINK_FEED_REGISTRY).latestRoundData(asset_,
    QUOTE_CURRENCY) returns (
219             uint80 roundIdChainlink,
220             int256 chainlinkPrice,
221             uint256 startedAtChainlink,
222             uint256 timeChainlink,
223             uint80 answeredInRoundChainlink
224         ) {
225             priceFeedDecimals =
    FeedRegistryInterface(CHAINLINK_FEED_REGISTRY).decimals(asset_,
    QUOTE_CURRENCY);
226             priceFeedPrice = chainlinkPrice;
227         } catch {
228             revert IPriceOracleMiddleware.UnsupportedAsset();
229         }
```

The code never verifies:

- Staleness via `timeChainlink/startedAtChainlink` against a heartbeat or maximum age;
- Round completeness via `answeredInRoundChainlink >= roundIdChainlink`;
- Non-zero timestamps for the returned round.

In production, registry hiccups, aggregator migrations, or feed pauses can surface old or incomplete answers that pass these missing checks and propagate a wrong USD price through the system.

Exploit Scenario

A fallback price from the Chainlink Feed Registry is accepted even if it is stale or from an incomplete round. An opportunistic trader can exploit this when the registry surfaces an outdated price (for example during a paused feed or aggregator migration), causing misvaluation across protocols that use this middleware.

Consider the following attack scenario:

- Bob integrates this middleware as the sole USD price source in his protocol;
- The Chainlink feed for the asset is paused or lagging, and the registry returns an old round with `answeredInRound` lower than `roundId` or a timestamp older than the heartbeat;
- Alice observes the stale price being propagated by `getAssetPrice` / `getAssetsPrices` and times a trade or action that becomes favorable only under the stale valuation;
- Bob's protocol executes using the stale price, resulting in incorrect collateralization, liquidations, or pricing, and Alice captures a risk-free profit.

Realistic impact:

- Under-collateralized positions can be minted or liquidations can be triggered improperly;
- Opaque PnL for vaults or price-based fees due to using stale inputs;
- Cross-asset arbitrage enabled by time-window price gaps.

Recommendation

Validate Chainlink responses before use and fail closed if validation is not met.

Minimum checks to add after `latestRoundData` and before using the answer:

- Require `answeredInRound >= roundId` to ensure the round is complete;
- Require `startedAt != 0` and `updatedAt != 0` (Chainlink registry returns both) and that `updatedAt` is within a configured staleness window (for example `block.timestamp - updatedAt <= maxDelay`);

- Require `price > 0` before any casts or conversions.

Suggested pattern inside `_getAssetPrice` after the registry call:

- Read `roundId`, `answer`, `startedAt`, `updatedAt`, `answeredInRound` from the registry;
- If any check fails, revert with a domain-specific error (for example `StalePrice` or `IncompleteRound`);
- Only then convert to `QUOTE_CURRENCY_DECIMALS`.

Also consider per-asset configuration of `maxDelay`/heartbeat, and emitting an event when a stale or incomplete round is detected to aid monitoring.

[Go back to Findings Summary](#)

H3: Receiver-directed deposits allow third parties to refresh a victim's redemption lock (DoS)

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Griefing

Description

Redemption locks are refreshed on deposits and mints for the address passed as `account_`. In `PlasmaVault`, the value forwarded as `account_` for deposit-like operations is the `receiver`, which can be any address supplied by the caller. Because ERC-4626 `deposit/assets, receiver)` allows depositing to arbitrary recipients, a malicious user can send dust deposits to a victim and continuously refresh the victim's lock without consent.

*Listing 7. Code snippet from
contracts/managers/access/RedemptionDelayLib.sol*

```
62         } else if (sig_ == DEPOSIT_SELECTOR || sig_ == MINT_SELECTOR || sig_
    == DEPOSIT_WITH_PERMIT_SELECTOR) {
63             IporFusionAccessManagersStorageLib.setRedemptionLocks(account_);
64         }
```

In `PlasmaVault._checkCanCall`, deposit-like calls pass the `receiver` as the `account_` to be updated, which need not equal the caller.

Listing 8. Code snippet from contracts/vaults/PlasmaVault.sol

```
1403         } else if (this.deposit.selector == sig || this.mint.selector ==
    sig) {
1404             (, address receiver) = abi.decode(_msgData()[4:], (uint256,
    address));
```

```

1405
1406         /// @dev check if the receiver of shares has access to deposit
    or mint and setup delay
1407         IporFusionAccessManager(authority()).canCallAndUpdate(receiver,
    address(this), sig);
1408         /// @dev check if the caller has access to deposit or mint and
    setup delay
1409         (immediate, delay) =
    AuthorityUtils.canCallWithDelay(authority(), caller_, address(this), sig);
1410     } else if (this.depositWithPermit.selector == sig) {
1411         (, address receiver,,,,) = abi.decode(_msgData()[4:], (uint256,
    address, uint256, uint8, bytes32, bytes32));
1412
1413         /// @dev check if the receiver of shares has access to
    depositWithPermit and setup delay
1414         IporFusionAccessManager(authority()).canCallAndUpdate(receiver,
    address(this), sig);

```

Effectively, any third party can refresh a victim's lock timestamp to `block.timestamp + REDEMPTION_DELAY_IN_SECONDS` by depositing a tiny amount to that victim as the `receiver`. While the lock is active, the victim cannot `withdraw`, `redeem`, or `transfer` shares; being the `from` in `transferFrom` is also blocked. Repeating dust deposits causes persistent denial-of-service.

Exploit Scenario

A malicious user can grief another user without their consent.

Consider the following attack scenario:

1. Alice funds the vault with a minimal amount of assets and calls `deposit(1, bob)` where `bob` is Bob's address;
2. The vault calls `canCallAndUpdate(receiver=bob, target=vault, selector=deposit)` which triggers `setRedemptionLocks(bob)`;
3. Bob attempts to `withdraw`, `redeem`, `transfer`, or be the `from` in `transferFrom`, but `lockChecks` reverts with `AccountIsLocked` while the lock is active; and
4. Alice repeats dust deposits to Bob to keep extending Bob's lock indefinitely, causing an ongoing denial-of-service for Bob's exits and

transfers.

Recommendation

Change who gets locked on deposit-like operations and/or require consent to be locked.

Fix the vulnerability by either:

- Updating the lock only for the deposit initiator: in `PlasmaVault._checkCanCall`, call `canCallAndUpdate(caller_, address(this), sig)` for `deposit`, `mint`, and `depositWithPermit` so that `lockChecks` sets the lock for the funder, not the arbitrary `receiver`;
- Requiring explicit consent from the `receiver` to accept third-party deposits that would lock their account (for example, an `acceptDeposit` flow or opt-in flag);
- Skipping lock updates when `receiver != caller_`, or enforcing a minimum `assets` threshold for lock updates to eliminate dust griefing; or
- If keeping receiver-based locking is a business requirement, add a per-account allowlist of trusted depositors to prevent unsolicited lock updates.

[Go back to Findings Summary](#)

H4: Iteration over hooks stops at first empty slot, skipping valid hooks after gaps

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/handlers/pre_hooks/pre_hooks/ExchangeRateValidatorPreHook.sol	Type:	Logic error

Description

The `ExchangeRateValidatorPreHook._runHooks` loop uses `break` when it encounters `address(0)` in the hooks array. Because `ExchangeRateValidatorConfigLib.parseConfigs` returns fixed-size arrays populated sparsely at declared indices (leaving unused positions as `address(0)`), any gap causes the loop to terminate early and skip all subsequent valid hooks.

This contradicts the developer comment that the loop “Skips empty entries” and can silently disable security- or accounting-related pre/post hooks placed after a gap.

Affected components:

- Function `ExchangeRateValidatorPreHook._runHooks` (called from `run` for both `preHooks` and `postHooks`).
- Library `ExchangeRateValidatorConfigLib.parseConfigs`, which produces fixed-size arrays with potential gaps.

Listing 9. Code snippet from

contracts/handlers/pre_hooks/pre_hooks/ExchangeRateValidatorPreHook.sol

```
93 function _runHooks(bytes4 selector_, Hook[] memory hooks_) private {
94     uint256 length = hooks_.length;
```

```

95     for (uint256 i; i < length; ++i) {
96         address implementation = hooks_[i].hookAddress;
97         if (implementation == address(0)) {
98             break;
99         }
100         Address.functionDelegateCall(implementation,
abi.encodeWithSelector(IPreHook.run.selector, selector_));
101     }
102 }

```

The **break** means the iteration stops at the first empty entry instead of continuing over later indices.

*Listing 10. Code snippet from
contracts/handlers/pre_hooks/pre_hooks/ExchangeRateValidatorConfigLib.s
ol*

```

152 function parseConfigs(bytes32[] memory configs_)
153     internal
154     pure
155     returns (Hook[] memory preHooks, Hook[] memory postHooks, ValidatorData
memory validationData, uint256 index)
156 {
157     index = type(uint256).max;
158
159     preHooks = new Hook[](MAX_PRE_HOOKS);
160     postHooks = new Hook[](MAX_POST_HOOKS);
161
162     uint256 configsLength = configs_.length;
163     ExchangeRateValidatorConfig memory config;
164     // Process configs: insert hooks at positions corresponding to their
index
165     for (uint256 i; i < configsLength; ++i) {
166         config = bytes32ToExchangeRateValidatorConfig(configs_[i]);
167         if (config.typ == HookType.PREHOOKS) {
168             Hook memory hook = bytes31ToHook(config.data);
169             if (hook.index >= MAX_PRE_HOOKS) {
170                 revert ExchangeRateValidatorConfigLibInvalidHookIndex();
171             }
172             if (preHooks[hook.index].hookAddress != address(0)) {
173                 revert ExchangeRateValidatorConfigLibHookPositionOccupied();
174             }
175             preHooks[hook.index] = hook;

```


By design, hooks are inserted at their declared `index` into fixed-size arrays, and unused positions remain `address(0)`. There is no guarantee of contiguity, therefore stopping at the first empty slot will omit valid hooks at higher indices.

Exploit Scenario

A configuration that contains gaps in the hook indices causes later hooks to be skipped, disabling intended safeguards.

Consider the following attack scenario:

1. Alice supplies a configuration where `preHooks[0]` is a benign hook, `preHooks[1]` is left empty, and a critical validation hook (for example, `ValidateAllAssetsPricesPreHook`) sits at `preHooks[2]`.
2. Alice triggers a vault operation that invokes `ExchangeRateValidatorPreHook.run`.
3. The contract enters `_runHooks(selector_, preHooks)` and executes index 0. Upon hitting `preHooks[1] == address(0)`, it executes `break` and terminates the loop.
4. The hook at index 2 is never executed, so expected validations or state updates (like price checks or balance updates) do not occur before the operation.
5. Bob's protocol processes the operation under missing controls, potentially allowing Alice to benefit from stale prices or un-updated balances and causing financial or accounting impact.

Recommendation

Replace the premature loop termination with skipping semantics, and/or return contiguous arrays from configuration parsing.

Recommended approaches:

- In `ExchangeRateValidatorPreHook._runHooks`, replace `break` with `continue` so empty entries are skipped without terminating the loop.
- Alternatively (or additionally), modify `ExchangeRateValidatorConfigLib.parseConfigs` to compact hooks into a dynamic array that contains only non-zero entries in execution order, and iterate across that array.
- Add invariants/tests ensuring that a gap in indices does not prevent later hooks from running.
- Align code and comments: if non-contiguous indices are allowed, the iterator must not assume that empties only appear at the end.

Security hardening:

- Enforce a configuration-time check that either forbids gaps or emits a clear error if a non-contiguous hook order is detected.
- Log a warning or revert when encountering a zero hook at an index lower than a previously seen non-zero index, if non-contiguous configurations are considered misconfiguration.

[Go back to Findings Summary](#)

H5: Performance fee shares over-minted due to missing normalization by current exchange rate

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/managers/fee /FeeManager.sol	Type:	Logic error

Description

The function `calculateAndUpdatePerformanceFee` computes the performance fee in shares using a high-water mark scheme but normalizes the gain delta by `10 ** assetDecimals_` instead of the current exchange rate. This treats an amount-denominated delta as if it were a price ratio, which systematically over-mints fee shares when the current exchange rate `P` is not equal to `10 ** assetDecimals_`.

Both `actualExchangeRate_` and the stored high-water mark `highWaterMark` are measured as `convertToAssets(10 ** decimals())`, i.e., assets obtainable for `10 ** shareDecimals` shares. Their difference `delta = P - H` is therefore in asset units for a fixed share quantity. To mint shares corresponding to a fee fraction `f` of the gain, the correct dilution-based formula is:

- `mintedShares = totalSupply * f * (P - H) / P`.

The implementation instead does:

- `sharesToHarvest = totalSupply * (P - H) / 10 ** assetDecimals` and then `feeShares = sharesToHarvest * f / 10000`.

When `P != 10 ** assetDecimals`, this mints too many shares by roughly a factor of `P / 10 ** assetDecimals`. For example, with `shareDecimals = assetDecimals = 18`, `P = 2e18`, `H = 1e18`, and `f = 20%`, the intended result is

$\text{totalSupply} * 0.2 * (1\text{e}18 / 2\text{e}18) = 0.1 * \text{totalSupply}$, but the implementation mints $\text{totalSupply} * (1\text{e}18 / 1\text{e}18) * 0.2 = 0.2 * \text{totalSupply}$ (2x over-minting).

Affected elements:

- Function `FeeManager.calculateAndUpdatePerformanceFee` (logic and arithmetic);
- Storage `HighWaterMarkPerformanceFeeStorage.highWaterMark` (units and updates);
- Call path `PlasmaVault._addPerformanceFee` ->
`PlasmaVaultFeesLib.prepareForAddPerformanceFee` ->
`FeeManager.calculateAndUpdatePerformanceFee` -> `_mint(recipient, feeShares)` to `PERFORMANCE_FEE_ACCOUNT`.

Listing 11. Code snippet from contracts/managers/fee/FeeManager.sol

```
529     function calculateAndUpdatePerformanceFee(  
530         uint128 actualExchangeRatepass:[_],  
531         uint256 totalSupplypass:[_],  
532         uint256 performanceFeepass:[_],  
533         uint256 assetDecimalspass:[_]  
534     ) external returns (address recipient, uint256 feeShares) {  
535         if (msg.sender != PLASMAPASS:[_]VAULT) {  
536             revert NotPlasmaVault();  
537         }  
538  
539         HighWaterMarkPerformanceFeeStorage memory highWaterMarkStorage =  
540             FeeManagerStorageLib.getPlasmaVaultHighWaterMarkPerformanceFee();  
541  
542         if (highWaterMarkStorage.highWaterMark == 0) {  
543             FeeManagerStorageLib.updateHighWaterMarkPerformanceFee(actualExchangeRatepas  
544                 s:[_]);  
545             return (address(0), 0);  
546         }  
547         if (actualExchangeRatepass:[_] <= highWaterMarkStorage.highWaterMark) {
```

```

548         if (
549             highWaterMarkStorage.updateInterval != 0
550             && block.timestamp >= highWaterMarkStorage.lastUpdate +
             highWaterMarkStorage.updateInterval
551         ) {
552             FeeManagerStorageLib.updateHighWaterMarkPerformanceFee(actualExchangeRatepass:[_]);
553         }
554         return (address(0), 0);
555     }
556
557     uint256 delta = actualExchangeRatepass:[_] -
uint256(highWaterMarkStorage.highWaterMark);
558     uint256 sharesToHarvest = Math.mulDiv(totalSupplypass:[_], delta, 10
** assetDecimalspass:[_]);
559     feeShares = Math.mulDiv(sharesToHarvest, performanceFee:pass:[_],
10000);
560
561     FeeManagerStorageLib.updateHighWaterMarkPerformanceFee(actualExchangeRatepass:[_]);
562     return (PERFORMANCEpass:[_]FEEpass:[_]ACCOUNT, feeShares);
563 }

```

Exploit Scenario

The vulnerability can be exercised during any action that triggers performance fee realization while the current exchange rate is above the stored high-water mark.

Consider the following attack scenario:

1. Alice, a beneficiary of `PERFORMANCE_FEE_ACCOUNT`, monitors when `convertToAssets(10 ** decimals())` has risen above the high-water mark `H`;
2. Alice triggers a vault operation that calls `_addPerformanceFee` (for example, calling `redeem(1, ...)` which is unrestricted for share owners);
3. `PlasmaVault._addPerformanceFee` calls `PlasmaVaultFeesLib.prepareForAddPerformanceFee`, passing `actualExchangeRate = convertToAssets(10 ** decimals())`;

4. `FeeManager.calculateAndUpdatePerformanceFee` computes `sharesToHarvest = totalSupply * (P - H) / 10 ** assetDecimals` and then `feeShares = sharesToHarvest * feeBps / 10000`, failing to divide by `P`;
5. The vault mints `feeShares` directly to `PERFORMANCE_FEE_ACCOUNT` via `_mint(recipient, feeShares)`;
6. Because `P > 10 ** assetDecimals` is typical during profitable periods, the minted amount is overstated (e.g., 2x when `P = 2e18` and `assetDecimals = 18`);
7. Alice proceeds to realize value from the over-minted shares, and
8. Bob, an ordinary depositor, is diluted more than the configured fee would intend.

Recommendation

Normalize the gain by the current exchange rate, not by `10 ** assetDecimals_`. Compute the fee shares directly as a proportion of the current price to avoid unit mismatch.

Recommended change in `calculateAndUpdatePerformanceFee`:

- Replace `sharesToHarvest = Math.mulDiv(totalSupply_, delta, 10 ** assetDecimals_)` with `sharesToHarvest = Math.mulDiv(totalSupply_, delta, uint256(actualExchangeRate_))`;
- Keep rounding down to avoid over-minting, then compute `feeShares = Math.mulDiv(sharesToHarvest, performanceFee_, 10000)`;
- Preserve the existing high-water mark update semantics.

Additional hardening and tests:

- Add unit tests that assert `feeShares == floor(totalSupply * feeBps/10000 * (P - H) / P)` for representative cases;

- Add invariant tests ensuring fee minting does not exceed `feeBps/10000` of unrealized gains valued at current `P`;
- Document the units of `actualExchangeRate` and `highWaterMark` in storage and function docs to prevent future regressions.

[Go back to Findings Summary](#)

H6: claimRewards allows arbitrary fuse delegatecalls, enabling privilege escalation from CLAIM_REWARDS_ROLE

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/managers/rewards/RewardsClaimManager.sol	Type:	Access control

Description

RewardsClaimManager.claimRewards only verifies that each `FuseAction.fuse` address is supported via `FusesLib.isFuseSupported` and then forwards the entire `FuseAction[]` to `PlasmaVault.claimRewards`.

In `PlasmaVault.claimRewards`, every action is executed using `Address.functionDelegateCall` with arbitrary `FuseAction.data`. There is no enforcement that the selector in `data` corresponds to a claim-specific entrypoint. This permits any caller holding `CLAIM_REWARDS_ROLE` on `RewardsClaimManager` to delegatecall into any supported fuse with an arbitrary selector and calldata in the Plasma Vault context.

This breaks the intended separation of duties between reward-claim operations and privileged market operations. A caller with the narrowly scoped claim role can perform actions that are normally restricted to higher-privileged flows (e.g., `execute`/Alpha flows), such as borrowing, supplying, or otherwise mutating protocol positions through non-claim fuse methods. This is a classic privilege escalation through an unbounded delegatecall surface.

*Listing 12. Code snippet from
contracts/managers/rewards/RewardsClaimManager.sol*

```
145 function claimRewards(FuseAction[] calldata calls_) external restricted {
146     uint256 len = calls_.length;
147
148     for (uint256 i; i < len; ++i) {
149         if (!FusesLib.isFuseSupported(calls_[i].fuse)) {
150             revert FusesLib.FuseUnsupported(calls_[i].fuse);
151         }
152     }
153
154     PlasmaVault(PLASMA_VAULT()).claimRewards(calls_);
155 }
```

Listing 13. Code snippet from contracts/vaults/PlasmaVault.sol

```
984 function claimRewards(FuseAction[] calldata calls_) external override
    nonReentrant restricted {
985     uint256 callsCount = calls_.length;
986     for (uint256 i; i < callsCount; ++i) {
987         calls_[i].fuse.functionDelegateCall(calls_[i].data);
988     }
989 }
```

Additionally, the set of "reward fuses" exposed by the manager is the global fuses list, not a reward-only subset. This further widens the attack surface by allowing any supported fuse to be used through the claim path.

*Listing 14. Code snippet from
contracts/managers/rewards/RewardsClaimManager.sol*

```
120 function getRewardsFuses() external view returns (address[] memory) {
121     return FusesLib.getFusesArray();
122 }
```

Exploit Scenario

A caller with only rewards-claim permissions can escalate privileges by

executing arbitrary fuse functions through the claim path.

Consider the following attack scenario:

1. Alice is granted `CLAIM_REWARDS_ROLE` on `RewardsClaimManager` for a vault;
2. Alice prepares a single `FuseAction` where `fuse` is the address of a supported market fuse (for example, `MoonwellBorrowFuse`) and `data` encodes a non-claim selector, e.g.,

```
abi.encodeWithSelector(MoonwellBorrowFuse.enter.selector,  
MoonwellBorrowFuseEnterData(asset, amount));
```
3. Alice calls `RewardsClaimManager.claimRewards(calls_)` with this crafted action;
4. The manager validates only that the `fuse` address is supported and forwards the call array to the vault;
5. `PlasmaVault.claimRewards` iterates and performs `delegatecall` to the fuse with Alice's arbitrary selector and calldata;
6. The vault executes a borrow/supply/exit function in the fuse context, mutating protocol positions without going through the higher-privileged `execute` flow;
7. As a result, the vault's risk/positioning can be maliciously altered (e.g., unintended borrowing, leverage changes, deallocation), potentially leading to losses, liquidations, or policy violations.

Recommendation

Constrain the claim path to claim-only intent at both the manager and vault levels.

Recommended changes:

- Whitelist allowed selectors per fuse for the claim path and enforce them before delegating. Extract `bytes4 selector = bytes4(calls_[i].data)` and

require that `selector` is one of the claim-specific functions for the given `fuse` (for example, `claim()`, `claimTransient()`, or other narrowly scoped reward-collection methods).

- Maintain a separate rewards-only fuse registry for `RewardsClaimManager` (do not reuse the global fuses array), and expose only this list via `getRewardsFuses()`.
- Alternatively, remove `PlasmaVault.claimRewards` and implement claims through a dedicated reward-collection facade that does not offer generic `delegatecall`, or route reward collection through the same `execute` pipeline with all validations and post-processing applied.
- Add unit tests that attempt to call non-claim selectors (e.g., `enter`, `exit`, `supply`, `borrow`) through the claim path and assert revert.

These changes preserve the separation of duties between reward collection and privileged market operations, preventing privilege escalation from `CLAIM_REWARDS_ROLE` to broad vault control.

[Go back to Findings Summary](#)

H7: Public canCallAndUpdate lets anyone set a victim's redemption lock without a real deposit

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Griefing

Description

`IporFusionAccessManager.canCallAndUpdate(caller, target, selector)` is `external` and unguarded. It calls `RedemptionDelayLib.lockChecks(caller, selector)` before delegating to `super.canCall`. Since `lockChecks` bases its behavior solely on the `selector` and a freely provided `account_`, any user can invoke `canCallAndUpdate(victim, anyTarget, depositSelector)` and refresh the victim's lock even when no deposit occurred on `target`.

*Listing 15. Code snippet from
contracts/managers/access/RedemptionDelayLib.sol*

```
53     function lockChecks(address account_, bytes4 sig_) internal {
54         if (
55             sig_ == WITHDRAW_SELECTOR || sig_ == REDEEM_SELECTOR || sig_ ==
TRANSFER_FROM_SELECTOR
56             || sig_ == TRANSFER_SELECTOR
57         ) {
58             uint256 unlockTime =
IporFusionAccessManagersStorageLib.getRedemptionLocks().redemptionLock[account_];
59             if (unlockTime > block.timestamp) {
60                 revert AccountIsLocked(unlockTime);
61             }
62         } else if (sig_ == DEPOSIT_SELECTOR || sig_ == MINT_SELECTOR || sig_
== DEPOSIT_WITH_PERMIT_SELECTOR) {
63             IporFusionAccessManagersStorageLib.setRedemptionLocks(account_);
64         }
```

`canCallAndUpdate` exposes this path to anyone and does not ensure that the caller is the `target` vault nor that the selector corresponds to a real, in-progress vault action.

*Listing 16. Code snippet from
contracts/managers/access/IporFusionAccessManager.sol*

```
163     function canCallAndUpdate(address caller_, address target_, bytes4
      selector_)
164         external
165         override
166         returns (bool immediate, uint32 delay)
167     {
168         RedemptionDelayLib.lockChecks(caller_, selector_);
169         return super.canCall(caller_, target_, selector_);
170     }
```

As a consequence, arbitrary third parties can set or extend redemption locks on victims by calling this function directly with a deposit-like selector.

Exploit Scenario

Any user can grief another without interacting with a vault deposit.

Consider the following attack scenario:

1. Alice calls `canCallAndUpdate(bob, target=vault, selector=PlasmaVault.deposit.selector)` on the Access Manager contract;
2. The Access Manager runs `lockChecks(bob, selector)` and sets Bob's lock via `setRedemptionLocks(bob)`;
3. No actual `deposit` occurred, but Bob is now locked and cannot `withdraw`, `redeem`, or `transfer` until the lock expires; and
4. Alice repeats the call periodically to keep Bob locked indefinitely.

Recommendation

Restrict who may call `canCallAndUpdate` and bind lock updates to genuine vault actions.

Fix the vulnerability by either:

- Enforcing `require(msg.sender == target_)` inside `canCallAndUpdate` so only the target vault can trigger stateful lock updates;
- Additionally verifying that `target_` is a registered or authorized vault (for example, checking a role on `target_`);
- Passing `target_` through to `lockChecks` and scoping state updates to the vault context, or moving the lock update to the vault itself after decoding and validating call parameters; or
- If external exposure is not required, make `canCallAndUpdate` `internal` / `private` and have vaults use `canCall` for view-only permission checks while performing lock updates locally.

[Go back to Findings Summary](#)

H8: Arbitrary external calls allow persistent ERC20 approvals from this contract to attacker-controlled spenders

High impact finding

Impact:	High	Confidence:	Medium
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorEth.sol	Type:	Trust model

Description

The `execute` dispatcher performs arbitrary external calls provided by the caller. Because these calls execute in the context of this contract, any user can invoke ERC20 `approve(spender, amount)` on arbitrary tokens on behalf of this contract. Such approvals persist across transactions and are not automatically cleared.

This enables pre-seeding unlimited allowances from this contract to attacker-controlled spenders. If the contract later holds those tokens (during a swap, or due to accidental transfers/refunds), the attacker can move them out via `transferFrom` without interacting with this contract again.

*Listing 17. Code snippet from
contracts/fuses/universal_token_swapper/SwapExecutorEth.sol*

```
74     for (uint256 i; i < len; ++i) {
75         if (data_.ethAmounts[i] > 0) {
76             Address.functionCallWithValue(data_.targets[i],
data_.callDatas[i], data_.ethAmounts[i]);
77         } else {
78             Address.functionCall(data_.targets[i], data_.callDatas[i]);
79         }
80     }
```

The loop calls arbitrary `targets` with arbitrary `callDatas` from this contract, enabling token `approve` calls to be made from the contract's address and remain in effect for future transactions.

Exploit Scenario

Consider the following attack scenario:

1. Alice calls `execute` with `targets = [USDC]` and `callDatas = [approve(alice_spender, type(uint256).max)]`, setting an unlimited allowance from the contract to `alice_spender`.
2. At a later time, Bob's workflow or an unrelated party causes USDC to be held by the contract (e.g., during a swap prior to sweeping, or via accidental transfer/refund).
3. Alice (as `alice_spender`) invokes `USDC.transferFrom(address(this), alice, balance)`, draining all USDC from the contract using the pre-established approval.
4. The contract has no mechanism to prevent or revoke this, so Alice can repeat this any time USDC appears on the contract.

Recommendation

Constrain or eliminate the ability to create persistent approvals from this contract.

Recommended hardening:

- Add an authorization layer: only accept intents signed by the `recipient`, and restrict `targets`/spenders to an allowlist of vetted routers.
- Enforce ephemeral approvals: when an approval is required, set `approve(spender, 0)` first, set `approve(spender, exactAmount)`, execute the call that pulls tokens, and then immediately reset the allowance to 0 in the same transaction.

- Deny by policy any direct ERC20 `approve` calls in `execute` unless the spender is an explicitly approved router and the token is the declared `tokenIn` for the call. Validate amounts against expected spend.
- Prefer push-based transfers (`safeTransfer` of `tokenIn` to the router) where possible to avoid granting pull allowances entirely.

Also consider periodic sweeping of nonzero allowances and a monitoring alert for unexpected allowance changes from this contract address.

[Go back to Findings Summary](#)

M1: withdraw transfers fewer assets than requested by subtracting fee from assets, violating ERC4626 withdraw semantics

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/vaults/PlasmaVault.sol	Type:	Standards violation

Description

`withdraw(assets, receiver, owner)` reduces the assets delivered to `receiver` by subtracting the asset-equivalent of fee shares, instead of burning additional fee shares while transferring exactly `assets` to the receiver.

Per ERC4626, the `assets` parameter to `withdraw` is the exact amount of underlying that must be sent to `receiver`. Any fee should be charged via additional shares burned from `owner` (or a separate accounting), not by decreasing the assets paid out.

Affected elements:

- Function `withdraw` in contract `PlasmaVault`.
- External dependency `WithdrawManager.canWithdrawFromUnallocated` produces `feeSharesToBurn`.
- The internal call `super._withdraw` is provided with `assets_ - convertToAssets(feeSharesToBurn)` as the payout amount, producing less than requested.

Listing 18. Code snippet from contracts/vaults/PlasmaVault.sol

```
714         uint256 shares = convertToShares(assets_);  
715
```

```

716         uint256 feeSharesToBurn =
WithdrawManager(withdrawManager).canWithdrawFromUnallocated(shares);
717
718         withdrawnShares = shares - feeSharesToBurn;
719
720         super._withdraw(
721             _msgSender(), receiver_, owner_, assets_ -
super.convertToAssets(feeSharesToBurn), withdrawnShares
722         );
723
724         if (feeSharesToBurn > 0) {
725             if (_msgSender() != owner_) {
726                 _spendAllowance(owner_, _msgSender(), feeSharesToBurn);
727             }
728
729             _burn(owner_, feeSharesToBurn);
730         }

```

The code pays `assets_ - convertToAssets(feeSharesToBurn)` to the `receiver`, and burns `feeSharesToBurn` separately, resulting in the receiver getting fewer assets than requested by the function argument. This diverges from the ERC4626 reference behavior and can break routers and applications relying on standard semantics.

Exploit Scenario

Routers and wallets assume that `withdraw(assets, receiver, owner)` transfers exactly `assets` to `receiver`.

Consider the following attack scenario:

1. Alice requests `withdraw(100, receiver, owner)` using Bob's ERC4626-compatible router that ensures the receiver must receive 100 units;
2. The vault computes `feeSharesToBurn > 0`, subtracts `convertToAssets(feeSharesToBurn)` from the payout, and transfers less than 100 to `receiver`;
3. Bob's router must either top-up the shortfall from its own funds to honor the guarantee, or the transaction reverts and degrades user experience;

4. Bob's protocol suffers loss or operational instability due to reliance on standard ERC4626 semantics that are violated here.

Recommendation

Charge withdrawal fees by burning additional shares, not by reducing the assets transferred.

Fix the vulnerability by either:

- Transferring exactly `assets_` to `receiver_` and burning `shares + feeSharesToBurn` from `owner_` as needed; or
- Refactoring to compute a single `sharesToBurn` that covers both the requested assets and the fee, and invoking `super._withdraw(msgSender, receiver_, owner_, assets_, sharesToBurn)`.

Add tests to assert that, for any non-reverting call, `withdraw(assets)` results in exactly `assets` sent to `receiver`, with fee charged only in shares.

[Go back to Findings Summary](#)

M2: divisionInt negates int256.min when taking absolute value, causing overflow and revert

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/libraries/math /IporeMath.sol	Type:	Arithmetics

Description

Function `divisionInt` computes absolute values by negating negative operands. Negating `type(int256).min` overflows in Solidity because there is no positive representation for the minimum two's-complement value. This makes the function revert for edge-case inputs.

Listing 19. Code snippet from contracts/libraries/math/IporeMath.sol

```
69 function divisionInt(int256 x_, int256 y_) internal pure returns (int256 z) {
70     uint256 absX_ = uint256(x_ < 0 ? -x_ : x_);
71     uint256 absY_ = uint256(y_ < 0 ? -y_ : y_);
```

If either `x_` or `y_` equals `type(int256).min`, the unary minus overflows and the call reverts before performing the division.

Exploit Scenario

Consider the following attack scenario:

1. Alice supplies inputs that propagate a sentinel value equal to `type(int256).min` into a flow that calls `divisionInt`.
2. Alice triggers the computation so that the negative operand is negated during absolute value calculation.
3. The negation overflows in Solidity 0.8.x and reverts immediately.

4. Bob's operation reverts consistently for that input pattern, enabling a denial of service on the affected path.

Recommendation

Handle the `int256` minimum explicitly when computing absolute values.

Fix the vulnerability by either:

- Guarding with `require(x_ != type(int256).min && y_ != type(int256).min)` if such inputs are unsupported; or
- Implementing an overflow-safe absolute value, for example: compute `abs = uint256(x_ >= 0 ? x_ : -(x_ + 1)) + (x_ < 0 ? 1 : 0)`, and similarly for `y_`.

Document the supported input domain and add tests that cover boundary values.

[Go back to Findings Summary](#)

M3: Unrestricted arbitrary external calls enable theft of contract-held tokens and persistent approvals

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/zaps/ERC4626 ZapIn.sol	Type:	Trust model

Description

The `zapIn` flow executes arbitrary, user-supplied external calls as this contract via `Address.functionCall` over `zapInData.calls`. With `msg.sender` equal to the zap contract, these calls can move any tokens currently held by the contract or set allowances from the contract to attacker-controlled spenders. There is no allowlist or selector-level filtering.

Immediate and deferred exfiltration are both possible:

- Immediate transfer-out: a call like `token.transfer(attacker, token.balanceOf(address(this)))` drains balances present on the contract at execution time.
- Persistent approvals: a call like `token.approve(attacker, type(uint256).max)` sets allowances from the contract that persist beyond the transaction. Later, the attacker can call `transferFrom` off-chain to drain any tokens that land on the contract.

Listing 20. Code snippet from contracts/zaps/ERC4626ZapIn.sol

```
92         results = new bytes[](callsLength);
93         for (uint256 i; i < callsLength; i++) {
94             results[i] =
                zapInData_.calls[i].target.functionCall(zapInData_.calls[i].data);
```

Notes:

- These calls run before the ERC4626 deposit and the subsequent refund loop, and they execute with the zap contract as `msg.sender`.
- The only bound checked (`minAmountToDeposit`) applies solely to the vault's `asset` amount, not to arbitrary token balances or approvals, so it does not prevent the exfiltration patterns above.

Exploit Scenario

Exfiltration via persistent approvals that are later used to drain funds.

Consider the following attack scenario:

1. Alice prepares a `zapIn` where `calls` includes multiple `approve(attacker, type(uint256).max)` invocations for widely held tokens (for example, USDC, WETH, DAI), targeting those token contracts.
2. Alice fetches just enough of the target vault `asset` via the allowance helper to meet `minAmountToDeposit`, ensuring the `zapIn` call succeeds.
3. The transaction executes; approvals from the zap contract to Alice's EOA are now in place and persist.
4. Hours or days later, Bob (or any user) accidentally transfers one of the approved tokens to the zap contract or leaves residual balances there via other flows.
5. Alice uses `transferFrom(zapContract, Alice, amount)` on those token contracts to pull the contract-held tokens using the approvals she planted earlier.

Recommendation

Constrain or remove the arbitrary call primitive so the zap contract cannot be

abused to move or approve its own balances.

Fix the vulnerability by either:

- Replacing the fully arbitrary `calls` array with a curated set of adapters/targets and validating both target addresses and function selectors against an allowlist; reject `approve`, `transfer`, and `transferFrom` selectors except to vetted destinations that are required by the zap flow.
- If arbitrary calls are required, maintain and enforce post-conditions: record allowances and balances of all touched tokens before executing calls and revert unless, at the end of the zap, all allowances from the zap contract to third parties are zero and no non-vault assets decreased unexpectedly.

Additional hardening:

- Prefer a pull-based token model where the zap contract holds no residual balances and grants no third-party allowances; use an intermediary like `ERC4626ZapInAllowance` or EIP-2612 permits from the user directly to trusted adapters instead of approvals from the zap.
- If approvals must be used, immediately revoke them after use and limit spenders to trusted contracts only.

[Go back to Findings Summary](#)

M4: convertWadToAssetDecimals overflows on scaling 10^{**}delta or on multiplication for extreme decimals

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/libraries/math /IpOrMath.sol	Type:	Arithmetics

Description

When converting from WAD to asset decimals, function `convertWadToAssetDecimals` multiplies by 10^{**}delta for `assetDecimals_ > 18`. In Solidity, 10^{**}delta reverts on overflow for `delta >= 78`, and even for smaller `delta`, the product `value_ * 10^{**}\text{delta}` can overflow `uint256` and revert for large `value_`.

Without input validation, extreme `assetDecimals_` or large `value_` can cause unexpected reverts and denial of service.

Listing 21. Code snippet from contracts/libraries/math/IpOrMath.sol

```
38 function convertWadToAssetDecimals(uint256 value_, uint256 assetDecimals_)
    internal pure returns (uint256) {
39     if (assetDecimals_ == WAD_DECIMALS) {
40         return value_;
41     } else if (assetDecimals_ > WAD_DECIMALS) {
42         return value_ * BASIS_OF_POWER ** (assetDecimals_ - WAD_DECIMALS);
43     } else {
44         return division(value_, BASIS_OF_POWER ** (WAD_DECIMALS -
            assetDecimals_));
45     }
46 }
```

In production deployments that whitelist well-behaved tokens, the usual `decimals <= 18` keeps the operation safe. The function still assumes inputs are

within safe ranges and lacks guarding for abnormal decimals.

Exploit Scenario

Consider the following attack scenario:

1. Alice ensures that a component processes a token configured with an extreme `decimals` value such that `assetDecimals_ - 18 >= 78`.
2. Alice triggers a conversion that calls `convertWadToAssetDecimals(value, assetDecimals_)` with a non-zero `value`.
3. The exponentiation `10**delta` overflows and reverts, or, for smaller but still large `delta`, the multiplication `value * 10**delta` overflows.
4. Bob's accounting flow reverts mid-execution, disrupting balance updates and blocking dependent operations.

Recommendation

Add explicit bounds and use safe multiplication for scaling.

Fix the vulnerability by either:

- Enforcing `assetDecimals_ - 18 <= 77` to avoid `10**delta` overflow, and also verifying `value_ <= type(uint256).max / 10**delta` before multiplying; or
- Using a safe mul-div utility (for example, `mulDiv` from a well-audited math library) to compute `value_ * 10**delta` with overflow checks.

Additionally, constrain supported asset decimals at integration boundaries so hostile tokens cannot introduce pathological parameters.

[Go back to Findings Summary](#)

M5: convertToWadInt cast of 10^{**}delta to int256 can revert for extreme decimals

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/libraries/math /IporeMath.sol	Type:	Arithmetics

Description

When `assetDecimals_` exceeds WAD (18), function `convertToWadInt` divides by a scaling factor computed as `int256(10**delta)`. Casting a large `uint256` to `int256` reverts in Solidity if the value exceeds `type(int256).max`.

Since 10^{**77} is greater than `type(int256).max`, any `delta >= 77` (for example, `assetDecimals_ >= 95`) will revert on the cast. For even larger deltas (`delta >= 78`), the exponentiation `10**delta` itself overflows `uint256` and reverts. Missing input validation makes the code fragile to extreme decimals sourced from tokens or composed decimals.

Listing 22. Code snippet from contracts/libraries/math/IporeMath.sol

```
52 function convertToWadInt(int256 value_, uint256 assetDecimals_) internal pure
    returns (int256) {
53     if (value_ == 0) {
54         return 0;
55     }
56     if (assetDecimals_ == WAD_DECIMALS) {
57         return value_;
58     } else if (assetDecimals_ > WAD_DECIMALS) {
59         return divisionInt(value_, int256(BASIS_OF_POWER ** (assetDecimals_ -
        WAD_DECIMALS)));
60     } else {
61         return value_ * int256(BASIS_OF_POWER ** (WAD_DECIMALS -
        assetDecimals_));
62     }
63 }
```

In production, typical ERC20 decimals and price feed decimals keep `delta` small. However, the function remains unsafe if an unexpected token reports an extreme `decimals` value or if composed decimals exceed safe bounds.

Exploit Scenario

Consider the following attack scenario:

1. Alice introduces or configures an asset with a manipulated `decimals` value such that `assetDecimals_ >= 95` is fed into conversion.
2. Alice triggers a code path that calls `convertToWadInt(value, assetDecimals_)` with `value != 0`.
3. The cast `int256(10**(assetDecimals_ - 18))` reverts due to exceeding `type(int256).max`, or the exponentiation itself overflows for even larger deltas.
4. Bob's balance computation reverts, preventing reporting or downstream processing for that market and causing a denial of service.

Recommendation

Harden input validation and avoid unsafe casts.

Fix the vulnerability by either:

- Enforcing an upper bound on `assetDecimals_` so that `assetDecimals_ - 18 <= 76` (to keep `10**delta <= type(int256).max`) and `assetDecimals_ - 18 <= 77` if exponentiation safety for `uint256` is also considered; and
- Replacing the `int256` cast with a safe conversion that checks the scaling factor against `type(int256).max` before casting.

Alternatively, refactor `divisionInt` to accept a `uint256` divisor and handle the sign from the numerator only, allowing the scaling factor to remain `uint256` and eliminating the problematic cast.

[Go back to Findings Summary](#)

M6: previewWithdraw divides by fee instead of applying discount $1/(1 - \text{fee})$, inflating required shares and breaking ERC4626 previews

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/vaults/PlasmaVault.sol	Type:	Standards violation

Description

`previewWithdraw` computes the required shares by dividing by the fee rate rather than applying a discount factor of $1e18 - \text{fee}$.

In ERC4626, `previewWithdraw(assets)` must return the number of shares that would be burned to withdraw exactly `assets` units of the underlying. If a withdrawal fee is expressed in WAD as `fee`, the correct required shares should scale as $\text{baseShares} * 1e18 / (1e18 - \text{fee})$. The implementation instead returns $\text{baseShares} * 1e18 / \text{fee}$, which explodes the estimate (for example, `fee = 5e16` yields ~20× larger result).

This causes severe preview inaccuracies and breaks off-chain quoting, slippage protection, and router integrations that rely on ERC4626 semantics. The function becomes especially dangerous for integrators that quote pre-trade numbers based on previews and then enforce user protections on-chain.

Affected elements:

- Function `previewWithdraw` in contract `PlasmaVault`.
- State and external dependencies: `WithdrawManager.getWithdrawFee` provides the WAD fee; `super.previewWithdraw` returns the baseline shares for `assets` before fee math.

Listing 23. Code snippet from contracts/vaults/PlasmaVault.sol

```
757     function previewWithdraw(uint256 assets_) public view override returns
      (uint256) {
758         address withdrawManager =
            PlasmaVaultStorageLib.getWithdrawManager().manager;
759
760         if (withdrawManager != address(0)) {
761             /// @dev get withdraw fee in shares with 18 decimals
762             uint256 withdrawFee =
                WithdrawManager(withdrawManager).getWithdrawFee();
763
764             if (withdrawFee > 0) {
765                 return Math.mulDiv(super.previewWithdraw(assets_), 1e18,
                    withdrawFee);
766             }
767         }
768         return super.previewWithdraw(assets_);
769     }
```

The highlighted return divides by `withdrawFee` rather than by `(1e18 - withdrawFee)`. This violates ERC4626 preview expectations and leads to inconsistent behavior across clients using previews for routing.

Exploit Scenario

An integration relies on `previewWithdraw` to determine how many shares a user must provide to receive a target amount of assets.

Consider the following attack scenario:

1. Alice configures or operates a system where a non-zero withdrawal fee (for example, 5%) is active;
2. Alice initiates a withdrawal through Bob's router that trusts `previewWithdraw` to compute the share amount;
3. The router requests far more shares than actually needed because `previewWithdraw` uses `1/fee` instead of `1/(1-fee)`; and

4. Bob's protocol either over-requests and fails user protections, or it must subsidize or re-route at a loss to honor quotes, creating operational and financial risk.

Recommendation

Replace the divisor with the discount factor and align the math with ERC4626 semantics.

Fix the vulnerability by either:

- Using `Math.mulDiv(super.previewWithdraw(assets_), 1e18, 1e18 - withdrawFee)` to apply the correct $1/(1 - fee)$ factor; or
- Delegating the fee math to a well-tested utility that mirrors the `previewRedeem` approach and adding invariants that `previewWithdraw` matches the actual `withdraw` path for non-reverting calls.

Add unit/property tests for:

- `withdrawFee == 0` (preview equals base);
- Small fees and boundary cases where `withdrawFee` approaches `1e18` (should clamp or revert); and
- Symmetry with `previewRedeem` and the actual `withdraw` execution path.

[Go back to Findings Summary](#)

M7: User-controlled sweeping of arbitrary token balances enables cross-user theft of stray funds

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/zaps/ERC4626 ZapIn.sol	Type:	Trust model

Description

After the ERC4626 deposit, `zapIn` iterates over user-supplied `assetsToRefundToSender` and transfers the contract's entire balance of each listed token to `currentZapSender`. There is no scoping to assets introduced by the current call and no per-user accounting. Any caller can therefore sweep unrelated balances that happen to be on the contract (accidental transfers, airdrops, donations, or leftovers).

Listing 24. Code snippet from contracts/zaps/ERC4626ZapIn.sol

```
106         uint256 assetsToRefundToSenderLength =
            zapInData_.assetsToRefundToSender.length;
107         address asset;
108         uint256 balance;
109
110         for (uint256 i; i < assetsToRefundToSenderLength; ++i) {
111             asset = zapInData_.assetsToRefundToSender[i];
112             balance = IERC4626(asset).balanceOf(address(this));
113             if (balance > 0) {
114                 IERC4626(asset).safeTransfer(currentZapSender, balance);
115             }
116         }
```

Technical impact:

- Cross-user theft of stray funds: the loop refunds total contract balances, not deltas attributable to the current `zapIn` caller.

- Violation of least privilege: a caller controls both the refund list and the recipient via `currentZapSender`.

Exploit Scenario

Sweeping of unrelated token balances using the user-controlled refund list.

Consider the following attack scenario:

1. Bob accidentally transfers 10,000 USDC to the zap contract address, leaving a balance on the contract.
2. Alice crafts a `zapIn` with `assetsToRefundToSender = [USDC]` and sets `minAmountToDeposit` just high enough to succeed.
3. The refund loop reads `USDC.balanceOf(address(this))` and transfers the entire 10,000 USDC to `currentZapSender` (Alice).
4. The transaction succeeds; Bob's funds are irreversibly swept to Alice without any link to her contribution in this call.

Recommendation

Restrict refunds to assets and amounts attributable to the current call, or remove user-controlled sweeping altogether.

Fix the vulnerability by either:

- Tracking pre- and post-call balances for each token listed in `assetsToRefundToSender`, and refunding only `delta = post - pre` to `currentZapSender`.
- Replacing the user-controlled sweep with an owner-only (or admin-gated) `rescueToken` for truly stranded funds, leaving no cross-user claim path.

Additional hardening:

- Maintain an allowlist of refundable assets directly tied to the zap

strategy; ignore arbitrary addresses supplied by callers.

- If the zap must support arbitrary tokens, require the caller to provide per-asset `minRefund` amounts based on pre-call snapshots and revert if `post < pre + minRefund`.

[Go back to Findings Summary](#)

M8: Monotonic nonce check allows high-nonce execution to invalidate lower nonces (violates sequential policy), enabling DoS/griefing

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/managers/context/ContextManager.sol	Type:	Denial of service

Description

The implementation enforces a monotonic nonce policy rather than the sequential nonce policy documented in the contract. The documentation states that nonces must be sequential and not reused, but the storage library accepts any `newNonce_` strictly greater than the current value. As a result, executing a message with a higher nonce will permanently invalidate all pending lower-nonce messages for the same `sender`.

This mismatch allows an untrusted executor to reorder execution and grief the system by burning earlier signed operations. It breaks FIFO semantics commonly assumed by off-chain orchestrators and can cause denial of service of intended actions until operators re-sign transactions with even higher nonces.

Listing 25. Code snippet from `contracts/managers/context/ContextManager.sol` (documented sequential nonces requirement)

```
270 /**
271
272  * @notice Executes multiple calls with signature verification and sender
    impersonation
273  * @param contextDataArray_ Array of context data containing signatures
274  * @return results Array of bytes containing the results of each call
275  * @dev Requirements:
```

```

276 * - Signatures must not be expired (block.timestamp <= expirationTime)
277 * - Signatures must be valid and match the claimed sender
278 * - Nonces must be sequential and not reused
279 * - Target addresses must be pre-approved
280 * - Array must not be empty
281 * - For each call:
282 *   1. Verifies signature and nonce
283 *   2. Sets up context with impersonated sender
284 *   3. Executes the call
285 *   4. Clears the context
286 * @custom:security Implements multiple security checks for signature-based
    execution
287 * @custom:access No role restrictions - callable by anyone with valid
    signatures
288 */
289
290 function runWithContextAndSignature(ContextDataWithSender[] calldata
    contextDataArray_)
291     external

```

*Listing 26. Code snippet from
contracts/managers/context/ContextManagerStorageLib.sol (monotonic
check accepts any higher nonce)*

```

122 /// @notice Verifies and updates the nonce for an address
123 /// @dev Ensures new nonce is higher than current nonce to prevent replay
    attacks
124 /// @param sender_ The address to update the nonce for
125 /// @param newNonce_ The new nonce value to set
126 /// @custom:throws NonceTooLow if the provided nonce is not higher than the
    current nonce
127 function verifyAndUpdateNonce(address sender_, uint256 newNonce_) internal {
128     Nonces storage $ = _getNonces();
129
130     if ($$.addressToNonce[sender_] >= newNonce_) {
131         revert NonceTooLow();
132     }
133
134     $$.addressToNonce[sender_] = newNonce_;
135
136     emit NonceUpdated(sender_, newNonce_);
137 }

```

Listing 27. Code snippet from contracts/managers/context/ContextManager.sol (use of the library during execution)

```
300 ContextDataWithSender calldata contextData;
301
302 for (uint256 i; i < length; ++i) {
303     contextData = contextDataArray_[i];
304
305     if (block.timestamp > contextData.expirationTime) {
306         revert SignatureExpired();
307     }
308
309     if (!_verifySignature(contextData)) {
310         revert InvalidSignature();
311     }
312
313     ContextManagerStorageLib.verifyAndUpdateNonce(contextData.sender,
        contextData.nonce);
314
315     results[i] = _executeWithinContext(contextData.target,
        contextData.sender, contextData.data);
316 }
```

Affected surface:

- Function `runWithContextAndSignature` updates nonce via `ContextManagerStorageLib.verifyAndUpdateNonce`.
- Library function `verifyAndUpdateNonce` accepts any strictly greater nonce.
- Storage struct `Nonces.addressToNonce` tracks the current nonce per `sender`.
- View helper `getNonce` exposes the current stored nonce.

Technical impact in production deployments:

- Executing a higher-nonce message first sets the stored nonce to that value, making all lower nonces permanently invalid (`NonceTooLow`).
- Off-chain systems assuming FIFO or strict sequencing can be grieved, causing operational failures, missed time-sensitive actions, or forcing

continual re-signing with ever-increasing nonces.

- While funds are not directly at risk, the denial-of-service vector is practical wherever multiple signed messages are in-flight or relayers are untrusted.

Exploit Scenario

A malicious or misbehaving executor can reorder valid messages and permanently invalidate earlier ones by submitting a higher-nonce message first.

Consider the following attack scenario:

1. Bob, an application operator, prepares three signed calls for `sender` with nonces 1, 2, and 3 and shares them with Alice (an untrusted relayer);
2. Alice submits the signed call with nonce 3 to `runWithContextAndSignature`;
3. The contract verifies the signature and calls `ContextManagerStorageLib.verifyAndUpdateNonce`, which sets `addressToNonce[sender] = 3` because 3 > current nonce;
4. Subsequent attempts to execute the earlier signatures with nonces 1 and 2 revert with `NonceTooLow` and can never succeed without new signatures, forcing Bob to reissue operations and causing disruption and potential economic loss (e.g., missed time windows, wasted gas, operational deadlocks).

Recommendation

Align the implementation with the documented sequential nonce policy, or explicitly adopt a multi-nonce scheme.

Fix the vulnerability by either:

- Enforcing strict sequential nonces: In

`ContextManagerStorageLib.verifyAndUpdateNonce`, require `newNonce_ == getNonce(sender_) + 1` (or equivalently check against `$.addressToNonce[sender_] + 1`) before updating the stored nonce. This guarantees FIFO processing and prevents a higher nonce from invalidating pending lower nonces.

- Designing an explicit multi-nonce window: If out-of-order execution is desired, maintain a bitmap or mapping of used nonces per `sender` (e.g., sliding window bitmaps) and validate each nonce independently, avoiding unintended invalidation of unrelated lower nonces.

Additionally:

- Update the documentation to match the chosen policy (sequential vs windowed nonces) and ensure comments in `ContextManager` accurately describe the guarantees.
- Add tests covering attempts to execute nonces out of order and verify that they behave according to policy (revert for sequential policy; succeed only once for multi-nonce policy).

[Go back to Findings Summary](#)

M9: divisionInt rounds negative half ties toward zero, violating nearest rounding

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/libraries/math/IporMath.sol	Type:	Logic error

Description

As implemented, function `divisionInt` is documented to round to the nearest integer, but its tie-breaking is asymmetric: negative results at exactly half a unit are rounded toward zero, whereas positive results are rounded away from zero. This violates true nearest rounding and introduces a directional bias whenever signed half-ties occur.

The branch uses a strict comparison for negatives and a non-strict comparison for positives. Therefore, `+2.5` becomes `3` but `-2.5` becomes `-2`.

Listing 28. Code snippet from contracts/libraries/math/IporMath.sol

```
69 function divisionInt(int256 x_, int256 y_) internal pure returns (int256 z) {
70     uint256 absX_ = uint256(x_ < 0 ? -x_ : x_);
71     uint256 absY_ = uint256(y_ < 0 ? -y_ : y_);
72
73     // Use bitwise XOR to get the sign on MBS bit then shift to LSB
74     // sign == 0x0000...0000 == 0 if the number is non-negative
75     // sign == 0xFFFF...FFFF == -1 if the number is negative
76     int256 sign = (x_ ^ y_) >> MSB;
77
78     uint256 divAbs;
79     uint256 remainder;
80
81     unchecked {
82         divAbs = absX_ / absY_;
83         remainder = absX_ % absY_;
84     }
85     // Check if we need to round
```

```

86     if (sign < 0) {
87         // remainder << 1 left shift is equivalent to multiplying by 2
88         if (remainder << 1 > absY_) {
89             ++divAbs;
90         }
91     } else {
92         if (remainder << 1 >= absY_) {
93             ++divAbs;
94         }
95     }
96
97     // (sign | 1) is cheaper than (sign < 0) ? -1 : 1;
98     unchecked {
99         z = int256(divAbs) * (sign | 1);
100     }
101 }

```

This logic makes exact half-ties increment for non-negative results only. In financial normalization code that uses `divisionInt` through `convertToWadInt`, a future change that passes signed intermediate values directly could accrue a small but systematic advantage to the side producing negative half-ties.

Exploit Scenario

Consider the following attack scenario:

1. Alice arranges inputs so that a downstream conversion calls `convertToWadInt` with a signed net amount where $y_ > 0$ and $x_$ is negative, and $\text{abs}(x_) / y_$ yields a fractional part of exactly 0.5 (for example, $x_ = -25, y_ = 10$).
2. Alice triggers many such operations near half-tie thresholds by fine-tuning amounts and decimals so half-ties occur repeatedly.
3. The contract rounds each negative half-tie toward zero due to the strict `>` comparison in the negative branch, returning `-2` instead of `-3` in the example.
4. Bob's protocol accrues a consistent 1-unit advantage for Alice per half-tie, creating measurable drift over large numbers of operations.

Recommendation

Adopt symmetric tie-breaking that matches the stated behavior.

Fix the vulnerability by either:

- Switching the negative branch comparison from $>$ to $\geq$ so half-ties round away from zero on both signs; or
- Implementing true nearest rounding explicitly using an offset before division, for example: for non-negative $x_$, compute $(absX_ + absY_ / 2) / absY_$; for negative $x_$, compute $(absX_ + absY_ / 2) / absY_$ then apply the sign.

Additionally, add unit tests covering half-ties for both positive and negative inputs to ensure the function's behavior matches the documentation.

[Go back to Findings Summary](#)

M10: Redemption lock is global per account across a shared Access Manager, enabling cross-vault DoS

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Logic error

Description

The redemption lock storage is a single mapping keyed only by `account`, maintained in the Access Manager's storage namespace. There is no scoping by vault (`target`) or by lock domain. Any vault wired to the same Access Manager instance will read and update the same per-account lock.

Listing 29. Code snippet from

contracts/managers/access/RedemptionDelayLib.sol

```
39     function getAccountLockTime(address account_) internal view returns
      (uint256) {
40         return
            IporFusionAccessManagersStorageLib.getRedemptionLocks().redemptionLock[account_];
41     }
```

The underlying storage uses a global mapping `redemptionLock[account]` without a `target` dimension.

Listing 30. Code snippet from

contracts/managers/access/IporFusionAccessManagersStorageLib.sol

```
11 struct RedemptionLocks {
12     /// @notice Maps user addresses to their deposit timestamp
```

```
13    /// @dev Used to enforce redemption delays after deposits
14    mapping(address account => uint256 depositTime) redemptionLock;
15 }
```

Impact: if multiple vaults share the same Access Manager, a deposit (or any lock-triggering call) affecting one vault will lock the account globally, blocking `withdraw`, `redeem`, `transfer`, and `transferFrom` checks across all vaults until the lock expires. This amplifies griefing impact and increases blast radius.

Exploit Scenario

A single malicious interaction in one vault locks the victim everywhere that uses the same Access Manager.

Consider the following attack scenario:

1. Alice performs a dust `deposit` to Bob as `receiver` in Vault-A, which uses Access Manager X;
2. Access Manager X updates `redemptionLock[bob]`;
3. Bob attempts to `withdraw` from Vault-B, which also uses Access Manager X; and
4. Vault-B's check reads the same global `redemptionLock[bob]` and reverts with `AccountIsLocked` until the lock expires. Alice can repeat the first step to keep Bob locked across both vaults.

Recommendation

Scope redemption locks by vault (target) instead of globally per account.

Fix the vulnerability by either:

- Changing storage to `mapping(address account => mapping(address target => uint256 lockUntil))` and updating `getAccountLockTime` /

`setRedemptionLocks` to read and write using the relevant `target` address;
pass `target_` into `lockChecks` so the correct scope is used;

- If storage changes are not possible, deploy a dedicated `IporFusionAccessManager` per vault so locks are not shared across unrelated vaults; or
- Introduce distinct lock namespaces per vault within the existing Access Manager by hashing `(account, target)` into the storage key.

[Go back to Findings Summary](#)

M11: totalAssets counts rewards held in RewardsClaimManager, inflating NAV and breaking convert/preview/limits

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/vaults/PlasmaVault.sol	Type:	Logic error

Description

`_getGrossTotalAssets` adds

`IRewardsClaimManager(rewardsClaimManagerAddress).balanceOf()` to the vault's own underlying balance and market-held assets. These rewards may be vested or otherwise pending in the RewardsClaimManager and are not held by the vault address yet.

When pending rewards are included in `totalAssets`, conversions and limits that use `totalAssets` are inflated relative to actually-available liquidity. This can misprice deposits/mints, overstate withdraw/redeem previews and `maxWithdraw/maxRedeem`, and cause fees to be computed on assets not under the vault's control.

Affected elements:

- Function `_getGrossTotalAssets` in contract `PlasmaVault`.
- Downstream functions relying on `totalAssets` (for example, `convertToShares`, `convertToAssets`, preview helpers, and limit functions).
- External dependency `IRewardsClaimManager.balanceOf` returns pending rewards external to the vault address.

Listing 31. Code snippet from contracts/vaults/PlasmaVault.sol

```
1376     function _getGrossTotalAssets() internal view returns (uint256) {
1377         address rewardsClaimManagerAddress =
            PlasmaVaultLib.getRewardsClaimManagerAddress();
1378
1379         if (rewardsClaimManagerAddress != address(0)) {
1380             return IERC20(asset()).balanceOf(address(this)) +
                PlasmaVaultLib.getTotalAssetsInAllMarkets()
1381
1382             +
                IRewardsClaimManager(rewardsClaimManagerAddress).balanceOf();
1383         }
1384
1385         return IERC20(asset()).balanceOf(address(this)) +
            PlasmaVaultLib.getTotalAssetsInAllMarkets();
1386     }
```

Including `IRewardsClaimManager(...).balanceOf()` overstates the vault's immediate asset base. As a result, a user may be quoted fewer shares on deposit, or previews may suggest larger withdrawals than can be satisfied until rewards are actually transferred into the vault.

Exploit Scenario

A malicious or misaligned operator can time the presence of pending rewards to influence conversions.

Consider the following attack scenario:

1. Alice, who controls the rewards flow, keeps a large amount vested in the RewardsClaimManager so that `balanceOf()` is high but tokens are not yet transferred to the vault address;
2. Bob deposits using on-chain conversions based on `totalAssets`, receiving fewer shares than fair value because the denominator counts assets not held by the vault yet;
3. Alice later moves the rewards into the vault, making share price appear

consistent again while Bob remains shorted on shares;

4. Bob suffers an economic loss due to mispriced deposit caused by inflated `totalAssets`.

Recommendation

Report only assets held by the vault and in live market positions in `totalAssets`. Do not include pending rewards that are not in the vault's custody.

Fix the vulnerability by either:

- Removing `IRewardsClaimManager(rewardsClaimManagerAddress).balanceOf()` from `_getGrossTotalAssets`, exposing a separate view (for example, `totalAssetsWithPendingRewards`) for UI purposes; or
- Transferring vested rewards to the vault before any operation that relies on `totalAssets` (for example, call `transferVestedTokensToVault` during fee realization or before conversions), ensuring conversions and limits only use settled balances.

Add invariant tests showing deposits/mints are priced solely by assets held by the vault and its markets, and that `maxWithdraw/maxRedeem` do not exceed liquid assets.

[Go back to Findings Summary](#)

M12: Unrestricted call loop can forward contract-held ETH and leverage existing approvals

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorEth.sol	Type:	Access control

Description

The dispatcher executes arbitrary external calls and, when `ethAmounts[i] > 0`, forwards ETH from this contract using `Address.functionCallWithValue`. There is no authorization or allowlisting of targets. If the contract ever holds ETH (e.g., from prior operations, refunds, or earlier steps in the same `execute`), a caller can forward that ETH to an arbitrary contract they control. In addition, any ERC20 approvals previously created from this contract can be exercised via crafted calls to approved spenders.

Listing 32. Code snippet from

contracts/fuses/universal_token_swapper/SwapExecutorEth.sol

```
74         for (uint256 i; i < len; ++i) {
75             if (data_.ethAmounts[i] > 0) {
76                 Address.functionCallWithValue(data_.targets[i],
77                 data_.callDatas[i], data_.ethAmounts[i]);
78             } else {
79                 Address.functionCall(data_.targets[i], data_.callDatas[i]);
80             }
```

The code forwards ETH value from this contract to arbitrary `targets` specified by the caller. It imposes no ownership or policy checks.

Exploit Scenario

Consider the following attack scenario:

1. Alice observes that the contract currently holds ETH (e.g., from an earlier operational mistake or a protocol refund).
2. Alice calls `execute` with `targets = [alice_contract]`, `ethAmounts = [balance]`, and a dummy `callData` that `alice_contract` can accept.
3. The dispatcher forwards the contract-held ETH to `alice_contract` via `functionCallWithValue`.
4. Alice keeps the ETH. The contract lacks any authorization checks to prevent this.

A related scenario abuses pre-existing approvals:

1. Alice previously used `execute` to set an unlimited allowance from this contract to a router she controls.
2. When the contract later holds the approved token, Alice includes a crafted call to that router in `execute` so the router `transferFrom`'s tokens out of the contract.
3. The tokens leave the contract before the end-of-function sweep.

Recommendation

Reduce the surface for unauthorized ETH and token movement during the call loop.

Recommended changes:

- If ETH forwarding is required for legitimate swaps, constrain it: only allow value to be sent to vetted routers, or require the call bundle to be authorized by a signed intent that names the allowed targets and value.
- Disallow arbitrary value forwarding when the contract holds ETH not

provided by the current bundle. Consider tracking per-execution value provenance and enforcing `sum(ethAmounts) == 0` for non-payable `execute`, or make `execute` payable and require `sum(ethAmounts) == msg.value`.

- Combine with the approval hardening: enforce ephemeral allowances limited to `tokenIn` and approved spenders, and revoke immediately after use.
- As a defense-in-depth, add an owner-only `recoverETH` function for accidental ETH only, and make public draining via the call loop impossible.

[Go back to Findings Summary](#)

L1: Stale or zero lastValidatedTimestamp returned: mapping is never updated nor cleared

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/managers/price/PriceOracleMiddlewareManagerLib.sol	Type:	Logic error

Description

The library exposes `getPriceValidationInfo` that returns `lastValidatedTimestamp` for a given `asset`, but the `lastValidatedTimestamps` mapping is never written anywhere and is not cleared when configuration is removed. As a result, integrators will always read `0` for newly configured assets, and any future code that writes to this mapping could yield stale values because `removePriceValidation` omits deleting the timestamp entry.

This breaks the intended time semantics of price validation and can mislead consumers that rely on the returned timestamp for freshness windows, risk throttling, or monitoring.

Listing 33. Code snippet from

contracts/managers/price/PriceOracleMiddlewareManagerLib.sol

```
13 struct PriceValidation {
14     /// @dev The price change is recorded in percentages where 1e18 = 100%.
15     mapping(address asset => uint256 maxPriceDelta) maxPriceDeltas;
16     mapping(address asset => uint256 lastValidatedPrice) lastValidatedPrices;
17     /// @dev Not implemented yet
18     mapping(address asset => uint256 lastValidatedTimestamp)
19     lastValidatedTimestamps;
20     EnumerableSet.AddressSet assets;
21 }
```

The getter returns the timestamp value that is never populated.

Listing 34. Code snippet from

contracts/managers/price/PriceOracleMiddlewareManagerLib.sol

```
179 function getPriceValidationInfo(address asset_)
180     internal
181     view
182     returns (uint256 maxPriceDelta, uint256 lastValidatedPrice, uint256
        lastValidatedTimestamp)
183 {
184     return (
185         _getPriceValidationSlot().maxPriceDeltas[asset_],
186         _getPriceValidationSlot().lastValidatedPrices[asset_],
187         _getPriceValidationSlot().lastValidatedTimestamps[asset_]
188     );
189 }
```

The validation routine updates only the baseline price, never the timestamp, even when the baseline is initialized or refreshed after a large move.

Listing 35. Code snippet from

contracts/managers/price/PriceOracleMiddlewareManagerLib.sol

```
211 uint256 lastValidatedPrice = priceValidation.lastValidatedPrices[asset_];
212 if (lastValidatedPrice == 0) {
213     priceValidation.lastValidatedPrices[asset_] = price_;
214     emit PriceValidationBaselineUpdated(asset_, price_);
215     return true;
216 }
217
218 uint256 priceDifference =
219     price_ > lastValidatedPrice ? price_ - lastValidatedPrice :
        lastValidatedPrice - price_;
220
221 uint256 priceDifferencePercent = Math.mulDiv(priceDifference, 1e18,
        lastValidatedPrice);
222
223 if (priceDifferencePercent > maxPriceDelta) {
224     revert PriceChangeExceeded(asset_, lastValidatedPrice, price_,
        maxPriceDelta);
225 }
226
```

```

227 if (priceDifferencePercent > maxPriceDelta / 2) {
228     priceValidation.lastValidatedPrices[asset_] = price_;
229     emit PriceValidationBaselineUpdated(asset_, price_);
230     return true;
231 }

```

When clearing configuration, the timestamp key is not deleted, which can leave stale values if another component ever writes it.

Listing 36. Code snippet from

contracts/managers/price/PriceOracleMiddlewareManagerLib.sol

```

155 function removePriceValidation(address asset_) internal {
156     if (asset_ == address(0)) {
157         revert AssetsAddressCanNotBeZero();
158     }
159
160     PriceValidation storage priceValidation = _getPriceValidationSlot();
161     priceValidation.assets.remove(asset_);
162
163     delete priceValidation.maxPriceDeltas[asset_];
164     delete priceValidation.lastValidatedPrices[asset_];
165
166     emit PriceValidationRemoved(asset_);
167 }

```

Exploit Scenario

An attacker can leverage the lack of timestamp updates to bypass or abuse any consumer logic that interprets the returned `lastValidatedTimestamp` as a freshness guarantee.

Consider the following attack scenario:

1. Alice observes that for any asset just configured by governance, `getPriceValidationInfo` returns a `lastValidatedTimestamp` of 0;
2. Alice interacts with Bob's protocol that expects a recent validation time to permit certain operations, but Bob's integration erroneously treats 0 as

”recent enough” or does not handle 0 specially;

3. Alice times interactions around large market moves so that price validation updates the baseline price without emitting any timestamp change; and
4. Bob’s risk checks that depend on the timestamp accept stale validations or fail open, leading to operations proceeding under outdated assumptions.

In practice, this can cause inconsistent behavior across components that rely on the timestamp for time-based guards, monitoring, SLAs, or circuit breakers. While not a direct fund-draining vector on its own, the inconsistency weakens defense-in-depth and can contribute to financial loss when combined with price manipulation or delayed oracle updates.

Recommendation

Implement clear, consistent semantics for `lastValidatedTimestamp` or remove it from the public surface.

Fix the vulnerability by either:

- Updating the timestamp whenever the baseline price is initialized or refreshed; and
- Clearing the timestamp when validation is removed.

A concrete approach:

- In `validatePriceChange`, set `priceValidation.lastValidatedTimestamps[asset_] = block.timestamp` in both branches that update `lastValidatedPrices`.
- In `removePriceValidation`, add `delete priceValidation.lastValidatedTimestamps[asset_];` to avoid stale data.
- Optionally, update `updatePriceValidation` to initialize the timestamp on first configuration if that matches intended semantics.

- If a timestamp is not required, remove it from `PriceValidation` and from the return of `getPriceValidationInfo` to avoid misleading consumers.

These changes align the read path with actual state transitions and prevent consumers from making incorrect freshness assumptions.

[Go back to Findings Summary](#)

L2: division docstring claims nearest rounding but implementation truncates toward zero

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/libraries/math/IporMath.sol	Type:	Code quality

Description

The function `division` is documented to round to the nearest integer, but it simply performs unsigned integer division, which truncates. The documentation contradicts the implementation and can mislead callers into expecting unbiased nearest rounding.

Listing 37. Code snippet from contracts/libraries/math/IporMath.sol

```
103 /// @notice Divides two uint256 numbers and rounds the result to the nearest
    integer
104 /// @param x_ The numerator
105 /// @param y_ The denominator
106 /// @return z_ The result of the division
107 function division(uint256 x_, uint256 y_) internal pure returns (uint256 z_)
    {
108     z_ = x_ / y_;
109 }
```

If a caller relies on the comment and expects nearest rounding, results will be consistently biased downward whenever there is a remainder.

Exploit Scenario

Consider the following attack scenario:

1. Alice crafts inputs so that a price or ratio computation uses `division` under the assumption of nearest rounding.

2. Alice chooses values where x_+ / y_+ leaves a remainder just below one half, maximizing downward truncation.
3. The contract applies truncated results in accounting, underpaying or undercrediting balances by small amounts per call.
4. Bob's protocol accumulates dust or minor shortfalls over time, diverging from expected nearest-rounded values.

Recommendation

Align code and documentation explicitly.

Fix the issue by either:

- Updating the docstring to state that the function truncates toward zero; or
- Implementing true nearest rounding, for example: $z_+ = (x_+ + y_+ / 2) / y_+$ for unsigned inputs, with tests covering half-ties.

Ensure all call sites are reviewed for their expected rounding semantics.

[Go back to Findings Summary](#)

L3: Unconditional substrates write persists entries under address(0) during removal and no-op updates

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/handlers/pre_hooks/PreHooksLib.sol	Type:	Logic error

Description

The library writes to `preHooksConfig.substrates[keccak256(abi.encodePacked(newImplementation, selector))]` unconditionally at the end of `setPreHookImplementations`. When `newImplementation` equals `address(0)` (removal) or both `oldImplementation` and `newImplementation` are `address(0)` (no-op), this statement stores a substrates array under the composite key derived from `(address(0), selector)`.

Affected function and storage:

- Function `setPreHookImplementations` in `contracts/handlers/pre_hooks/PreHooksLib.sol`.
- Storage inside `PlasmaVaultStorageLib.PreHooksConfig`: mappings `hooksImplementation` and `substrates`, and arrays/mappings `selectors` and `indexes`.

Why this is vulnerable:

- Removal path (`oldImplementation != address(0)` and `newImplementation == address(0)`): the code correctly removes the selector from `selectors`, cleans up `indexes`, and deletes the `substrates` entry for `(oldImplementation, selector)`. However, it then unconditionally writes a new `substrates` entry under `(address(0), selector)`, leaving stale

configuration attached to a disabled hook.

- Zero-to-zero path (`oldImplementation == address(0)` and `newImplementation == address(0)`): none of the conditional branches execute, yet the unconditional write still persists `substrates` under `(address(0), selector)` for a selector that is not registered in `selectors`, creating an orphaned entry.

Impact in production:

- Storage pollution and inconsistent cleanup semantics for disabled or non-existent pre-hooks.
- Potential confusion for off-chain tooling, tests, or future code that might read `getPreHookSubstrates(selector, address(0))` as a proxy for "disabled" state. Such consumers would observe non-empty configuration for a disabled hook.
- Unnecessary event emission and storage writes on true no-ops, increasing gas and complicating audit trails.

This behavior contradicts the stated intent to "properly clean up storage on removal" and creates unreachable configuration that current readers do not use.

*Listing 38. Code snippet from
contracts/handlers/pre_hooks/PreHooksLib.sol*

```
116             // If this is a new selector, add it to the array and update its
            index
117             if (oldImplementation == address(0) && newImplementation !=
                address(0)) {
118                 preHooksConfig.selectors.push(selector);
119                 preHooksConfig.indexes[selector] =
                    preHooksConfig.selectors.length - 1;
120                 preHooksConfig.substrates[
                    keccak256(abi.encodePacked(newImplementation, selector))] = substrates_[i];
121             }
122             // If we're removing an implementation, swap and pop from the
```

```

selectors array
123         else if (oldImplementation != address(0) && newImplementation ==
address(0)) {
124             uint256 index = preHooksConfig.indexes[selector];
125             uint256 lastIndex = preHooksConfig.selectors.length - 1;
126             if (index != lastIndex) {
127                 bytes4 lastSelector =
preHooksConfig.selectors[lastIndex];
128                 preHooksConfig.selectors[index] = lastSelector;
129                 preHooksConfig.indexes[lastSelector] = index;
130             }
131             preHooksConfig.selectors.pop();
132             delete preHooksConfig.indexes[selector];
133             delete preHooksConfig.substrates[
keccak256(abi.encodePacked(oldImplementation, selector))];
134         } else if (newImplementation != oldImplementation) {
135             delete preHooksConfig.substrates[
keccak256(abi.encodePacked(oldImplementation, selector))];
136         }
137
138         preHooksConfig.hooksImplementation[selector] =
newImplementation;
139         preHooksConfig.substrates[
keccak256(abi.encodePacked(newImplementation, selector))] = substrates_[i];
140
141         emit PreHookImplementationChanged(selector, newImplementation,
substrates_[i]);

```

Concrete technical evidence:

- Unconditional writes at lines 138–139 assign `hooksImplementation[selector] = newImplementation` and `substrates[hash(newImplementation, selector)] = substrates_[i]` regardless of branch outcome.
- In the removal branch, `delete preHooksConfig.substrates[hash(oldImplementation, selector)]` occurs first, then the unconditional write reintroduces a `substrates` entry under `(address(0), selector)`.
- In the zero-to-zero path, no branch runs, but the unconditional write still persists an entry under `(address(0), selector)` while the selector is absent from `selectors`.

Exploit Scenario

A privileged caller can repeatedly poison storage with orphaned substrates arrays under (`address(0)`, `selector`) by exercising the removal and no-op paths.

Consider the following attack scenario:

1. Alice holds the governance role that can call the vault's `setPreHookImplementations` and submits an update for selector `S` with `newImplementation = address(0)` and a non-empty `substrates` value;
2. The library removes `S` from `selectors`, deletes the (`oldImplementation`, `S`) entry, but then unconditionally stores `substrates` under (`address(0)`, `S`);
3. Later, Alice submits a no-op update for an unrelated selector `T` where both `oldImplementation` and `newImplementation` are `address(0)` but supplies arbitrary `substrates` data;
4. The library again persists `substrates` under (`address(0)`, `T`) even though `T` is not registered in `selectors`;
5. Bob, who maintains off-chain monitoring and automation, queries `getPreHookSubstrates(S, address(0))` and `getPreHookSubstrates(T, address(0))` to validate removals and sees non-empty data, leading to false positives, wasted operational time, and potential misconfiguration in automated pipelines.

Recommendation

Guard the final `substrates` assignment and avoid writing configuration for `address(0)` implementations. Make the write conditional on `newImplementation != address(0)` and keep deletion-only semantics for removals and true no-ops.

Fix the vulnerability by either:

- Move the `substrates` write into the relevant branches and only execute it when `newImplementation != address(0)`; or
- Add a guard before the unconditional write. Specifically, if `newImplementation == address(0)`, do not write to `substrates` for `(address(0), selector)`. If `newImplementation != address(0)` and `newImplementation != oldImplementation`, first delete the old key then write the new key. If `newImplementation == oldImplementation`, update `substrates` in-place for the existing non-zero key.

Additionally:

- Skip emitting `PreHookImplementationChanged` when `oldImplementation == newImplementation` to avoid noisy, misleading events; and
- Add tests asserting that `getPreHookSubstrates(selector, address(0))` returns an empty array after removals and no-ops, and that no `(address(0), selector)` entries are persisted.

[Go back to Findings Summary](#)

L4: Integer division in 1% tolerance check permits up to ~1.99% deviation

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol	Type:	Logic error

Description

The tolerance gate in `createAndAddPtTokenPriceFeed` is implemented with integer division and a `> 1` comparison on a percentage computed as `(abs(actual - expected) * 100) / expected`. Because Solidity truncates toward zero, deviations up to 1.99% evaluate to `1` and are accepted, even though the intent is to allow at most 1.00%.

Affected elements:

- Function: `createAndAddPtTokenPriceFeed`
- Local variables: `priceDelta`, `priceDeltaPercentage`

Listing 39. Code snippet from

contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol

```
158     if (price < expextedPriceAfterDeployment_) {
159         int256 priceDelta = expextedPriceAfterDeployment_ - price;
160         int256 priceDeltaPercentage = (priceDelta * 100) /
expextedPriceAfterDeployment_;
161
162         if (priceDeltaPercentage > 1) {
163             revert IPriceOracleMiddleware.PriceDeltaTooHigh();
164         }
165     } else {
166         int256 priceDelta = price - expextedPriceAfterDeployment_;
167         int256 priceDeltaPercentage = (priceDelta * 100) /
```

```

    expectedPriceAfterDeployment_;
168
169         if (priceDeltaPercentage > 1) {
170             revert IPriceOracleMiddleware.PriceDeltaTooHigh();
171         }

```

Example: $\text{expected} = 100$, $\text{actual} = 98.02 \rightarrow \text{delta} = 1.98$. The expression $(1.98 * 100) / 100$ truncates to 1, so the check does not revert even though the deviation is ~1.98%.

This weakens the intended safety guard for initializing a new PT price feed and can admit feeds with nearly 2% initial mispricing.

Exploit Scenario

A price feed that deviates by up to ~1.99% from the expected value can still be added because the integer-division check underestimates the deviation.

Consider the following attack scenario:

- Bob uses `createAndAddPtTokenPriceFeed` to bootstrap a new PT feed with a 1% tolerance policy;
- The external market has drifted, and the initial round for the PT is ~1.8% off the expected value;
- Alice, anticipating favorable mispricing once the feed is accepted, submits orders in Bob's protocol that will benefit from the ~1.8% valuation error;
- Bob's governance/operator runs the initializer; the check incorrectly passes; the feed is added; Alice's queued actions execute at a mispriced rate, capturing value.

Realistic impact:

- One-time initialization mispricing up to nearly 2% when a feed is added;
- Potentially unfair minting, redemption, or position opening during the

acceptance window.

Recommendation

Eliminate truncation error and express the threshold in basis points to preserve precision.

A robust approach is to compare products instead of dividing, or to use `Math.mulDiv` with rounding:

- Compute `absDeltaBp = Math.mulDiv(uint256(abs(price - expected)), 10_000, uint256(expected))` and revert if `absDeltaBp > 100` for a strict 1.00% maximum; or
- Avoid division entirely: revert if `uint256(abs(price - expected)) * 10_000 > uint256(expected) * 100`.

If the intent is to reject deviations strictly greater than 1.00% while allowing exactly 1.00%, use a `> 100` basis-point threshold. If the intent is to reject 1.00% and above, use `>= 100`.

Also normalize types early (`require(expected > 0)`, cast to `uint256`) and document the exact tolerance semantics to avoid ambiguity.

[Go back to Findings Summary](#)

L5: clearContext does not detect missing context due to getSenderFromContext fallback, allowing no-op clears and a misleading ContextCleared event

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/managers/context/ContextClient.sol	Type:	Logic error

Description

The function `clearContext` determines context presence by reading the effective sender via `ContextClientStorageLib.getSenderFromContext`. This helper falls back to `msg.sender` when no context is stored, thus the subsequent `currentSender == address(0)` check never triggers in normal execution. As a result, the function does not revert when the context is unset, contrary to the documented requirement, and it emits `ContextCleared` with the effective sender (typically `msg.sender`) rather than the previously stored context value.

This behavior both violates the interface requirement that an active context must exist before clearing and can mislead off-chain monitoring and audit trails by emitting a `ContextCleared` event even when nothing was set.

*Listing 40. Code snippet from
contracts/managers/context/ContextClient.sol*

```
65 function clearContext() external override restricted {
66     address currentSender = ContextClientStorageLib.getSenderFromContext();
67
68     if (currentSender == address(0)) {
69         revert ContextNotSet();
70     }
```

```

71
72     ContextClientStorageLib.clearContextStorage();
73
74     emit ContextCleared(currentSender);
75 }

```

The check relies on `getSenderFromContext`, which returns `msg.sender` when storage is empty, making `currentSender == address(0)` unreachable in practice.

*Listing 41. Code snippet from
contracts/managers/context/ContextClientStorageLib.sol*

```

72 function getSenderFromContext() internal view returns (address) {
73     address sender = getContextSender();
74
75     if (sender == address(0)) {
76         return msg.sender;
77     }
78
79     return sender;
80 }

```

The interface also states that clearing requires an active context.

*Listing 42. Code snippet from
contracts/managers/context/IContextClient.sol*

```

32 /**
33
34  * @notice Clears the current active context
35  * @dev Requirements:
36  * - Must be called by an authorized contract
37  * - An active context must exist
38  * - Emits ContextCleared event on successful cleanup
39  * @custom:security Should always be called after context operations are
    complete
40 */
41
42 function clearContext() external;

```

Exploit Scenario

An authorized operator can induce misleading `ContextCleared` events and bypass the expected revert on missing context.

Consider the following attack scenario:

1. Alice, an authorized context manager, ensures that no context is set (the stored `contextSender` is `address(0)`).
2. Alice calls `clearContext` directly; inside the function, `getSenderFromContext` returns `msg.sender` (Alice) due to fallback behavior.
3. The condition `currentSender == address(0)` does not hold; the function proceeds, re-writing the zero value and emitting `ContextCleared(Alice)`.
4. Bob, who relies on off-chain monitoring or expects a revert when no context is active, observes a valid-looking `ContextCleared` event and may draw incorrect conclusions about prior context usage or sequencing.

Realistic impact in production:

- Off-chain audit trails and monitoring can be polluted with `ContextCleared` events that never corresponded to an active context, hindering investigations and compliance checks.
- Any caller logic or orchestration assuming a revert on missing context will not detect mis-sequenced calls, potentially masking operator mistakes. While access is restricted by `restricted`, the behavior violates the contract's documented semantics and can impede operational safety tooling.

Recommendation

Use the raw stored context to check presence and to populate the event, avoiding the fallback to `msg.sender`.

Fix the vulnerability by either:

- Checking storage directly via `ContextClientStorageLib.isContextSenderSet()` (or `getContextSender()`), reverting when unset; and
- Emitting the previously stored context value (not the effective sender) in `ContextCleared`.

Concrete guidance:

- Replace `address currentSender = ContextClientStorageLib.getSenderFromContext();` with reading the stored value, for example `address stored = ContextClientStorageLib.getContextSender();`
- Revert if `stored == address(0)` with `ContextNotSet()`.
- Call `ContextClientStorageLib.clearContextStorage()`.
- Emit `ContextCleared(stored)` so logs reflect the actual cleared context.

This preserves the interface requirement that an active context must exist before clearing and ensures emitted events are accurate.

[Go back to Findings Summary](#)

L6: Stale mapping entries in `configureInstantWithdrawalFuses` cause permanent storage bloat when reducing or reordering the fuse list

Low impact finding

Impact:	Low	Confidence:	High
Target:	<code>./contracts/libraries/PlasmaVaultLib.sol</code>	Type:	Gas optimization

Description

The library stores per-entry parameters for instant withdrawals in a mapping keyed by `keccak256(abi.encodePacked(fuse, index))`. During reconfiguration, the implementation deletes and rewrites parameters only for keys derived from the new configuration. When the new fuse list is shorter or reordered relative to the old one, keys corresponding to removed or shifted indices from the previous configuration are never deleted, leaving unreachable dynamic arrays in storage. Over multiple reconfigurations this leads to cumulative storage bloat and higher long-term gas footprint for maintenance, upgrades, or migrations.

The mapping keys for previous `fuse` and `index` pairs are not swept, because the code only computes keys for the new array content. Since mappings are non-iterable, those orphaned entries cannot be enumerated later for cleanup without retaining the old fuse list in memory prior to replacement.

Listing 43. Code snippet from `contracts/libraries/PlasmaVaultLib.sol`

```
556 function
    configureInstantWithdrawalFuses(InstantWithdrawalFusesParamsStruct[]
    calldata fuses_) internal {
557     address[] memory fusesList = new address[](fuses_.length);
558
```

```

559     PlasmaVaultStorageLib.InstantWithdrawalFusesParams storage
instantWithdrawalFusesParams =
560         PlasmaVaultStorageLib.getInstantWithdrawalFusesParams();
561
562     bytes32 key;
563
564     for (uint256 i; i < fuses_.length; ++i) {
565         if (!FusesLib.isFuseSupported(fuses_[i].fuse)) {
566             revert FusesLib.FuseUnsupported(fuses_[i].fuse);
567         }
568
569         fusesList[i] = fuses_[i].fuse;
570         key = keccak256(abi.encodePacked(fuses_[i].fuse, i));
571
572         delete instantWithdrawalFusesParams.value[key];
573
574         for (uint256 j; j < fuses_[i].params.length; ++j) {
575             instantWithdrawalFusesParams.value[key].push(fuses_[i].params[j]);
576         }
577     }
578
579     delete PlasmaVaultStorageLib.getInstantWithdrawalFusesArray().value;
580
581     PlasmaVaultStorageLib.getInstantWithdrawalFusesArray().value =
fusesList;
582
583     emit InstantWithdrawalFusesConfigured(fuses_);
584 }

```

The getter shows how parameters are later retrieved using the same composite key, which means any entries whose **fuse** or **index** do not exist in the current array become permanently unreachable and thus stale:

Listing 44. Code snippet from contracts/libraries/PlasmaVaultLib.sol

```

513 function getInstantWithdrawalFusesParams(address fuse_, uint256 index_)
internal view returns (bytes32[] memory) {
514     return
PlasmaVaultStorageLib.getInstantWithdrawalFusesParams().value[keccak256(abi.
encodePacked(fuse_, index_))];
515 }

```

Exploit Scenario

A realistic scenario that demonstrates the impact on state hygiene and costs:

- Alice, an authorized governance operator, configures an instant withdrawal path with 20 entries; parameters are stored at keys `keccak256(fuse_i, i)` for `i` in `[0..19]`.
- Alice later reduces the configuration to 5 entries. The implementation deletes and overwrites only keys for the new 5 indices, leaving keys for indices 5..19 untouched in the `InstantWithdrawalFusesParams.value` mapping.
- Alice repeats growth-then-shrink cycles, causing unbounded accumulation of unreachable `bytes32[]` arrays in storage.
- Bob, a user, continues interacting with the vault. While functionality is unaffected, the protocol's state grows permanently, making future governance operations (like upgrades or migrations) more gas-expensive and operationally risky due to larger state to manage.

Recommendation

Delete parameter entries for the previous configuration before replacing the fuse array, so removed indices do not leave stale storage behind.

Fix the vulnerability by either:

- Reading the previous fuse list into memory and sweeping all old keys; or
- Introducing a configuration version and namespacing keys by version, then sweeping the old version.

A minimal, direct approach in the current design:

- Before writing `fusesList`, fetch the old array: `address[] memory prev = PlasmaVaultStorageLib.getInstantWithdrawalFusesArray().value;`

- For `uint256 i = 0; i < prev.length; ++i`, compute `bytes32 key = keccak256(abi.encodePacked(prev[i], i))`; and delete `PlasmaVaultStorageLib.getInstantWithdrawalFusesParams().value[key]`;
- Then proceed to write the new array and new parameters.

Operational safeguards:

- If the old list can be very large, consider imposing a governance-controlled maximum length, or allow chunked cleanup across multiple transactions to stay within block gas limits.

[Go back to Findings Summary](#)

W1: Supply cap unit mismatch: documentation claims underlying units while enforcement uses shares

Impact:	Warning	Confidence:	High
Target:	contracts/vaults/PlasmaVaultBase.sol	Type:	Code quality

Description

The `cap` getter is documented as "stored in underlying asset decimals" and its return value is described as "in underlying asset decimals". However, every enforcement point compares `cap` directly against `totalSupply()` (shares) and all ERC-4626 limits derive from the remaining share capacity. In this codebase, shares are not a 1:1 representation of the underlying amount: conversions incorporate a share-scale multiplier `10 ** DECIMALS_OFFSET` during bootstrap and use dynamic ratios thereafter. As a result, configuring `cap` using underlying units (as suggested by the comments) can make the effective limit off by a fixed factor and produce premature deposit failures or unexpected headroom.

Affected components and their unit assumptions:

- Function `cap` in `PlasmaVaultBase` exposes the raw storage value without unit conversion;
- Function `_update` in `PlasmaVaultBase` enforces `totalSupply()` (shares) $\leq$ `cap` on mint path;
- Functions `maxDeposit` and `maxMint` in `PlasmaVault` compute remaining capacity purely in shares and, for `maxDeposit`, convert that residual share space to assets only at the end;
- Governance functions `getTotalSupplyCap`/`setTotalSupplyCap` in `PlasmaVaultGovernance` annotate the value as "underlying asset decimals",

which conflicts with the above.

This inconsistency introduces a realistic misconfiguration risk. For example, with `DECIMALS_OFFSET = 2`, initial mints create `assets * 100` shares. If governance sets a cap based on underlying units, the cap will be 100x too small in the enforcement domain and deposits will fail far earlier than intended. Conversely, future changes to scaling assumptions would cause the opposite effect. The code is functionally consistent when treating `cap` as a share limit; only the documentation mislabels the unit.

Listing 45. Code snippet from `contracts/vaults/PlasmaVaultBase.sol`

```
110    /// @notice Gets the maximum total supply cap for the vault
111    /// @dev Retrieves the configured supply cap from ERC20CappedStorage
112    ///
113    /// Supply Cap System:
114    /// - Enforces maximum vault size limit
115    /// - Stored in underlying asset decimals
116    /// - Critical for deposit control
117    /// - Part of risk management
118    ///
119    /// Storage Pattern:
120    /// - Uses PlasmaVaultStorageLib.ERC20CappedStorage
121    /// - Single value storage slot
122    /// - Set during initialization
123    /// - Modifiable by governance
124    ///
125    /// Integration Context:
126    /// - Used during minting operations
127    /// - Referenced in deposit validation
128    /// - Part of supply control system
129    /// - Critical for vault scaling
130    ///
131    /// Use Cases:
132    /// - Deposit limit validation
133    /// - Share minting control
134    /// - TVL management
135    /// - Risk parameter monitoring
136    ///
137    /// Related Components:
138    /// - ERC20 Implementation
139    /// - Supply Control System
```

```

140    /// - Deposit Validation
141    /// - Risk Management
142    ///
143    /// @return uint256 The maximum total supply cap in underlying asset
    decimals
144    /// @custom:access Public view
145    /// @custom:security Non-privileged view function
146    function cap() public view virtual returns (uint256) {
147        return PlasmaVaultStorageLib.getERC20CappedStorage().cap;
148    }

```

Listing 46. Code snippet from contracts/vaults/PlasmaVaultBase.sol

```

237    /// @dev Notice! Can be executed only by Plasma Vault in delegatecall.
238    /// Combines the logic required for ERC20VotesUpgradeable and
    ERC20VotesUpgradeable
239    function _update(address from_, address to_, uint256 value_) internal
    virtual override {
240        super._update(from_, to_, value_);
241
242        /// @dev total supply cap validation is disabled when performance
    and management fee is minted
243        if (PlasmaVaultLib.isTotalSupplyCapValidationEnabled()) {
244            /// @dev check total supply cap
245            if (from_ == address(0)) {
246                uint256 maxSupply = cap();
247                uint256 supply = totalSupply();
248                if (supply > maxSupply) {
249                    revert ERC20ExceededCap(supply, maxSupply);
250                }
251            }
252        }
253
254        _transferVotingUnits(from_, to_, value_);
255    }

```

*Listing 47. Code snippet from
contracts/vaults/PlasmaVaultGovernance.sol*

```

1450    /// @notice Sets the total supply cap for the vault
1451    /// @dev Updates the vault's total supply limit while enforcing
    validation rules
1452    ///
1453    /// Supply Cap System:

```

```

1454    /// - Enforces maximum vault size
1455    /// - Controls total value locked (TVL)
1456    /// - Guards against excessive concentration
1457    /// - Supports gradual scaling
1458    ///
1459    /// Validation Requirements:
1460    /// - Must be non-zero value
1461    /// - Must be sufficient for operations
1462    /// - Should consider asset decimals
1463    /// - Must accommodate fee minting
1464    ///
1465    /// Storage Updates:
1466    /// - Updates PlasmaVaultStorageLib.ERC20CappedStorage
1467    /// - Stores cap in underlying asset decimals
1468    /// - Affects deposit availability
1469    /// - Impacts share minting limits
1470    ///
1471    /// Integration Context:
1472    /// - Used during deposit validation
1473    /// - Affects share minting operations
1474    /// - Part of risk management system
1475    /// - Critical for vault scaling
1476    ///
1477    /// Security Considerations:
1478    /// - Only callable by ATOMIST_ROLE
1479    /// - Critical for vault size control
1480    /// - Affects deposit availability
1481    /// - Impacts risk parameters
1482    ///
1483    /// Related Components:
1484    /// - ERC4626 Implementation
1485    /// - Deposit Control System
1486    /// - Fee Minting Logic
1487    /// - Risk Management Framework
1488    ///
1489    /// @param cap_ The new total supply cap in underlying asset decimals
1490    /// @custom:access ATOMIST_ROLE restricted
1491    /// @custom:security Critical for vault capacity management
1492    function setTotalSupplyCap(uint256 cap_) external override restricted {
1493        PlasmaVaultLib.setTotalSupplyCap(cap_);
1494    }

```

Exploit Scenario

This issue manifests through configuration rather than a reentrancy or

access-control break. Because documentation labels the cap as "underlying asset decimals" while enforcement uses shares, operators can be misled into setting incorrect values that materially change deposit behavior.

Consider the following attack scenario:

1. Alice, holding governance permissions, sets `setTotalSupplyCap(1_000_000)` thinking in underlying units (per the comments);
2. Alice deposits 10,000 underlying into an empty vault; due to `DECIMALS_OFFSET = 2`, the mint path produces about 1,000,000 shares for 10,000 assets;
3. The contract then enforces `totalSupply() > cap()` and reverts future deposits much earlier than the intended underlying limit;
4. Bob, an external user or integrator, experiences a denial-of-service on deposits and operational disruption until governance corrects the cap.

A mirrored failure can also occur when integrators and risk systems rely on the "underlying decimals" wording, causing faulty capacity dashboards and automated allocators to make incorrect routing decisions.

Recommendation

Standardize the cap to a single, explicit unit and make the code self-documenting to prevent future drift between implementation and comments.

Fix the vulnerability by either:

- Treating `cap` as a share-domain limit everywhere and updating all comments and NatSpec accordingly (e.g., "cap is denominated in shares"); or
- Exposing an asset-denominated governance API that converts to shares internally and stores the share-domain `cap` (e.g., accept `capAssets` in `setTotalSupplyCap`, convert with `convertToShares` under conservative

rounding, and persist the result), while keeping enforcement strictly in shares.

Additional hardening:

- Rename getters/setters or add explicit unit suffixes, such as `getTotalSupplyCapShares` and `setTotalSupplyCapShares`;
- Emit events and unit tests asserting that `cap >= _SHARE_SCALE_MULTIPLIER()` to avoid zero-cap traps at bootstrap;
- Add invariant documentation that `totalSupply()` is compared against cap in shares, and that `maxDeposit`/`maxMint` derive from remaining shares.

[Go back to Findings Summary](#)

W2: Direct call to `getCallbackHandlers(address[],bytes4[])` reads wrong storage context and returns incorrect data

Impact:	Warning	Confidence:	High
Target:	contracts/readers/CallbackHandlerReader.sol	Type:	Code quality

Description

The batch reader `getCallbackHandlers(address[] senders_, bytes4[] sigs_)` is designed to be executed via the `UniversalReader` pattern so that the code runs in the `PlasmaVault` storage context using `delegatecall`. When it is invoked directly on `CallbackHandlerReader`, it looks up keys in the namespaced `CALLBACK_HANDLER` storage slot within the reader's own storage and therefore returns `address(0)` for every key.

The function computes the same keys as `CallbackHandlerLib` and fetches values from the callback handler mapping. Without the `UniversalReader.read` indirection, the mapping is not the vault's mapping and will be empty in this contract.

Listing 48. Code snippet from `contracts/readers/CallbackHandlerReader.sol`

```
66 function getCallbackHandlers(address[] calldata senders_, bytes4[] calldata
    sigs_)
67     external
68     view
69     returns (address[] memory handlers)
70 {
71     if (senders_.length != sigs_.length) {
72         revert CallbackHandlerReaderInvalidArrayLength();
73     }
74 }
```

```

75     handlers = new address[](senders_.length);
76
77     for (uint256 i; i < senders_.length; ++i) {
78         bytes32 key = keccak256(abi.encodePacked(senders_[i], sigs_[i]));
79         handlers[i] =
            CallbackHandlerLib.getCallbackHandlerStorage().callbackHandler[key];
80     }
81 }

```

Realistic impact:

- Any integrator or on-chain consumer that calls this batch function directly will get an array of zeros even if handlers are properly configured in the target **PlasmaVault**.
- Monitoring and configuration tooling may produce false negatives or trigger unnecessary configuration changes.
- This is a correctness hazard in the read path and not a mutation bug, but it can cause operational mistakes downstream.

Exploit Scenario

Consider the following attack scenario:

1. Alice's monitoring bot queries `getCallbackHandlers(address[], bytes4[])` on the **CallbackHandlerReader** contract directly, passing several known `(sender, sig)` pairs.
2. The bot receives an array of `address(0)` values because the mapping is read from the reader's empty storage.
3. Assuming handlers are missing, Alice initiates maintenance actions or blocks user flows that rely on these callbacks.
4. Bob's protocol experiences avoidable downtime or misconfiguration churn caused by decisions based on incorrect read data.

Recommendation

Reduce the chance of misuse and make the correct path the only path:

- Remove or make `internal` the direct batch variant `getCallbackHandlers(address[], bytes4[])` and keep only `getCallbackHandlers(address plasmaVault_, address[] senders_, bytes4[] sigs_)` for external use.
- If you must keep the direct variant, add clear NatSpec warnings that it must be executed via UniversalReader (delegatecall in a `PlasmaVault` context). Optionally, introduce a runtime environment check that reverts on direct calls.
- Provide SDK/helper functions that always route reads through `UniversalReader.read` so integrators do not accidentally call the unsafe variant.

[Go back to Findings Summary](#)

W3: Direct call to `getCallbackHandler(address,bytes4)` reads wrong storage context and returns incorrect handler

Impact:	Warning	Confidence:	High
Target:	contracts/readers/CallbackHandlerReader.sol	Type:	Code quality

Description

The function `getCallbackHandler(address sender_, bytes4 sig_)` is intended to be executed via the UniversalReader pattern so that it runs in the storage context of a `PlasmaVault` using `delegatecall`. When it is called directly on `CallbackHandlerReader`, it reads from the namespaced storage slot `CALLBACK_HANDLER` in the reader's own storage rather than in the target `PlasmaVault`, which causes the function to return `address(0)` (or stale) values for all keys.

This occurs because `CallbackHandlerLib.getCallbackHandlerStorage()` returns a struct that is anchored to a fixed storage slot in whatever contract context the code runs. A direct call executes in the reader's context, which never populates that slot.

*Listing 49. Code snippet from
contracts/readers/CallbackHandlerReader.sol*

```
54 function getCallbackHandler(address sender_, bytes4 sig_) external view
    returns (address handler) {
55     bytes32 key = keccak256(abi.encodePacked(sender_, sig_));
56     handler =
        CallbackHandlerLib.getCallbackHandlerStorage().callbackHandler[key];
57 }
```

The storage indirection is implemented in `PlasmaVaultStorageLib` through a constant slot and an accessor that binds the mapping to that slot for the

current execution context. When the function above is executed via `delegatecall` from a `PlasmaVault`, the mapping is read from the vault's storage; when called directly, it is read from the reader's storage.

*Listing 50. Code snippet from
contracts/libraries/PlasmaVaultStorageLib.sol*

```
683 bytes32 private constant CALLBACK_HANDLER =  
    0xb37e8684757599da669b8aea811ee2b3693b2582d2c730fab3f4965fa2ec3e00;
```

*Listing 51. Code snippet from
contracts/libraries/PlasmaVaultStorageLib.sol*

```
1025 function getCallbackHandler() internal pure returns (CallbackHandler  
    storage handler) {  
1026     assembly {  
1027         handler.slot := CALLBACK_HANDLER  
1028     }  
1029 }
```

Realistic impact:

- Off-chain or on-chain integrators that accidentally call `getCallbackHandler(address, bytes4)` on the reader contract will retrieve incorrect information and may conclude that no handler is configured.
- Tooling or monitors that rely on this endpoint without the `UniversalReader` indirection can make wrong operational decisions (for example, attempt to configure handlers unnecessarily, surface false alarms, or assume callbacks will revert with `HandlerNotFound`).
- The bug does not change on-chain state by itself, but it is a correctness hazard for read paths and an integration footgun.

Exploit Scenario

Consider the following attack scenario:

1. Alice integrates with the vault and, by mistake, queries `getCallbackHandler(address, bytes4)` directly on `CallbackHandlerReader` instead of using the `UniversalReader`-enabled overload.
2. Alice receives `address(0)` as the handler because the mapping is read in the reader's storage context, which is empty.
3. Believing that no handler is set, Alice configures her system to execute flows that require no callback handler or triggers a governance proposal to "fix" the configuration.
4. Bob's protocol experiences failed or delayed operations when callbacks are expected, because decisions were based on incorrect read data.

Recommendation

Prefer design changes that eliminate the possibility of misuse and make the correct usage self-evident:

- Remove the externally callable variant `getCallbackHandler(address, bytes4)` and expose only the overload that accepts `plasmaVault_` and internally uses `UniversalReader.read`.
- If keeping the function is required for backwards compatibility, clearly document in NatSpec that it MUST only be executed via `UniversalReader` (delegatecall in a `PlasmaVault` context). Consider emitting a revert on direct calls by adding a lightweight runtime environment check specific to the vault context; alternatively, gate the direct reader behind a dedicated interface that is only reachable through `UniversalReader`.
- Provide a single entry point for external users, for example `getCallbackHandler(address plasmaVault_, address sender_, bytes4 sig_)`, to reduce the chance of accidentally reading the wrong storage context.

These changes ensure that all external reads unambiguously operate in the intended storage context.

[Go back to Findings Summary](#)

W4: Timestamp downcast to uint32 in updateManagementFeeData will revert after 2106, bricking fee-related code paths

Impact:	Warning	Confidence:	High
Target:	./contracts/libraries/PlasmaVaultLib.sol	Type:	Arithmetics

Description

The function downcasts `block.timestamp` to `uint32`. Once UNIX time exceeds $2^{32} - 1$ (circa February 2106), `SafeCast.toUint32` reverts, making all code paths that call this function permanently fail. This is a latent reliability defect that will DoS deposits, withdrawals, and any operations that realize or update management fees after the cutoff date.

Listing 52. Code snippet from contracts/libraries/PlasmaVaultLib.sol

```
447 function updateManagementFeeData() internal {
448     PlasmaVaultStorageLib.ManagementFeeData storage feeData =
        PlasmaVaultStorageLib.getManagementFeeData();
449     feeData.lastUpdateTimestamp = block.timestamp.toUint32();
450 }
```

The storage type enforces `uint32`, confirming the downcast and the revert condition when time exceeds the 32-bit range:

Listing 53. Code snippet from contracts/libraries/PlasmaVaultStorageLib.sol

```
962 struct ManagementFeeData {
963     address feeAccount;
964     uint16 feeInPercentage;
965     uint32 lastUpdateTimestamp;
966 }
```

Call sites demonstrate that this function is executed during initialization and fee realization, making the failure systemic across critical flows:

- `contracts/vaults/PlasmaVault.sol:1254`
- `contracts/vaults/extensions/WrappedPlasmaVaultBase.sol:1036`
- `contracts/vaults/lib/PlasmaVaultFeesLib.sol:58`

Exploit Scenario

Consider the following sequence after the UNIX timestamp passes $2^{32} - 1$ (~Feb 2106):

- Alice, a regular user, calls `deposit`, which triggers management fee realization and a timestamp update;
- The call reaches `updateManagementFeeData`, which executes `block.timestamp.toUint32()`;
- `SafeCast` reverts due to `block.timestamp > type(uint32).max`, causing the entire transaction to revert; and
- Bob and all other users can no longer deposit, withdraw, or realize management fees; the vault's fee-related flows are bricked until a migration or code change.

Recommendation

Widen the timestamp type to avoid the year-2106 cutoff and remove the failing cast.

Fix the vulnerability by either:

- Changing `ManagementFeeData.lastUpdateTimestamp` to `uint48` or `uint64`, and assigning `feeData.lastUpdateTimestamp = uint64(block.timestamp)`; (no `SafeCast` needed); or

- If storage layout must remain compatible, introduce a new field (for example, `uint64 lastUpdateTimestampV2`) and read/write the wider field going forward, while ignoring the old one.

Additional guidance:

- `uint48` is a common, space-efficient choice in modern libraries (safe for many trillions of years). If packing with other fields, ensure struct layout still aligns as intended.
- Verify all arithmetic that uses the timestamp (differences and comparisons) remains within 256-bit math and will not overflow.

[Go back to Findings Summary](#)

W5: Documentation mismatch: canWithdrawFromUnallocated reserves only released shares, not all pending requests

Impact:	Warning	Confidence:	High
Target:	./contracts/managers/withdraw/WithdrawManager.sol	Type:	Code quality

Description

The documentation of function `canWithdrawFromUnallocated` claims that Unallocated Balance is "vault's total balance - sum of all pending withdrawal requests". However, the implementation reserves only the amount of shares that were explicitly released via `releaseFunds` (tracked as `sharesToRelease`). Pending withdrawal requests that have not yet been released are not accounted for in the enforcement logic.

This creates a documentation-to-code divergence that can mislead integrators, auditors, or off-chain risk tooling. A caller reading the documentation could assume that unallocated withdrawals are bounded by the need to preserve all pending requests, while in reality only the "released" tranche is protected. Before funds are released, `sharesToRelease` equals 0, allowing unallocated withdrawals to consume the entire convertible balance even if many pending requests exist.

Listing 54. Abridged docstring describing Unallocated Balance from contracts/managers/withdraw/WithdrawManager.sol

```
102 /**
103
104  * @notice Validates and calculates withdrawal fee for unallocated balance
105  * @dev Only callable by PlasmaVault contract (TECH_PLASMA_VAULT_ROLE)
106  *
```

```

107
108 * Unallocated Balance:
109 * - Represents the portion of vault's assets not committed to pending
    withdrawal requests
110 * - Calculated as: vault's total balance - sum of all pending withdrawal
    requests
111 * - Available for immediate withdrawals without scheduling
112 * - Subject to different fee structure than scheduled withdrawals
113 * - Can be accessed through standard withdraw/redeem operations
114 *
115
116 * Validation Flow:
117 * 1. Balance Verification
118 *   - Checks PlasmaVault's total underlying token balance
119 *   - Subtracts total shares pending for scheduled withdrawals
120 *   - Ensures withdrawal amount + pending releases <= total unallocated
    balance
121 *   - Prevents double-allocation of shares

```

Listing 55. Core implementation that only reserves `sharesToRelease` in `contracts/managers/withdraw/WithdrawManager.sol`

```

156 function canWithdrawFromUnallocated(uint256 shares_) external restricted
    returns (uint256) {
157     address plasmaVaultAddress = msg.sender;
158     uint256 feeRate = WithdrawManagerStorageLib.getWithdrawFee();
159     uint256 balanceOfPlasmaVault =
    ERC4626(ERC4626(plasmaVaultAddress).asset()).balanceOf(plasmaVaultAddress);
160     uint256 plasmaVaultBalanceOfUnallocatedShares =
    ERC4626(plasmaVaultAddress).convertToShares(balanceOfPlasmaVault);
161     uint256 sharesToRelease =
    WithdrawManagerStorageLib.getSharesToRelease();
162
163
164     if (sharesToRelease > 0 && plasmaVaultBalanceOfUnallocatedShares <
    sharesToRelease + shares_) {
165         revert WithdrawManagerInvalidSharesToRelease(
166             sharesToRelease, shares_, plasmaVaultBalanceOfUnallocatedShares
167         );
168     }
169     if (feeRate > 0) {
170         //@dev 1e18 is the precision of the fee rate
171         return Math.mulDiv(shares_, feeRate, 1e18);
172     }
173     return 0;

```

```
174 }
```

Listing 56. For reference, `sharesToRelease` is set only by `releaseFunds` in `contracts/managers/withdraw/WithdrawManagerStorageLib.sol`

```
209 function getLastReleaseFundsTimestamp() internal view returns (uint256) {
210     return _getReleaseFunds().lastReleaseFundsTimestamp;
211 }
212
213 function getSharesToRelease() internal view returns (uint256) {
214     return uint256(_getReleaseFunds().sharesToRelease);
215 }
216
217 /// @notice Updates the last funds release timestamp
218 /// @param newReleaseFundsTimestamp_ New release funds timestamp to set
219 /// @param sharesToRelease_ Amount of funds released
220 function releaseFunds(uint256 newReleaseFundsTimestamp_, uint256
    sharesToRelease_) internal {
221     ReleaseFunds storage releaseFundsLocal = _getReleaseFunds();
222
223     uint256 lastReleaseFundsTimestamp =
        releaseFundsLocal.lastReleaseFundsTimestamp;
224
225     if (lastReleaseFundsTimestamp > newReleaseFundsTimestamp_) {
226         revert WithdrawManagerInvalidTimestamp(lastReleaseFundsTimestamp,
            newReleaseFundsTimestamp_);
227     }
228
229     releaseFundsLocal.lastReleaseFundsTimestamp =
        newReleaseFundsTimestamp_.toUint32();
230     releaseFundsLocal.sharesToRelease = sharesToRelease_.toUint128();
231
232     emit ReleaseFundsUpdated(newReleaseFundsTimestamp_.toUint32(),
        sharesToRelease_.toUint128());
233 }
```

Affected scope includes:

- Function `canWithdrawFromUnallocated` in contract `WithdrawManager`.
- Variable `sharesToRelease` maintained in storage by `WithdrawManagerStorageLib`.

- Integration points in contract `PlasmaVault` that call `canWithdrawFromUnallocated` during `withdraw` and `redeem` operations.

Realistic impact in production:

- Operators or integrators could plan liquidity assuming pending requests are always reserved, leading to unexpected shortfalls when unallocated withdrawals are permitted to proceed before `releaseFunds`.
- Users with pending (but not yet released) requests may experience delays or failures due to insufficient immediate liquidity after unallocated withdrawals consume the balance.
- Off-chain monitoring and risk calculations that rely on the documented definition may mis-report available liquidity, causing governance or automation errors.

Exploit Scenario

A misinterpretation of the documentation can be exploited to drain unallocated liquidity before scheduled requests are honored, because only `sharesToRelease` is reserved and pending requests are ignored until `releaseFunds` is called.

Consider the following attack scenario:

1. Bob submits large withdrawal requests that remain pending (not yet released by governance via `releaseFunds`).
2. Alice observes that `sharesToRelease` is currently 0 and calls the vault's `withdraw` to pull assets through the unallocated path, which invokes `canWithdrawFromUnallocated`.
3. The call succeeds because the check only enforces `plasmaVaultBalanceOfUnallocatedShares >= sharesToRelease + shares_`, and `sharesToRelease == 0`.

4. Alice's withdrawals consume most of the vault's immediately available balance.
5. Later, `releaseFunds` is called for Bob's requests; when Bob attempts `redeemFromRequest`, the vault must source assets from markets or will face insufficient immediate liquidity, risking delays or failed execution depending on downstream integrations and market conditions.

Recommendation

Choose one of the following mutually exclusive remedies, depending on intended protocol semantics:

Option A — Align documentation with the implementation:

- Clearly state that Unallocated Balance only reserves `sharesToRelease` (the tranche explicitly released by `releaseFunds`) and does not subtract the sum of all pending withdrawal requests.
- Update the docstring of `canWithdrawFromUnallocated` and any related references in `PlasmaVault` to remove language about subtracting pending requests.

Option B — Align implementation with the current documentation:

- Track an aggregated `totalPendingRequestShares` in storage, updated in `updateWithdrawRequest`, `decreaseSharesFromWithdrawRequest`, and `deleteWithdrawRequest` to avoid iterating over mappings.
- In `canWithdrawFromUnallocated`, enforce that `plasmaVaultBalanceOfUnallocatedShares >= totalPendingRequestShares + shares_` (and optionally also add `sharesToRelease` if double-accounting is not desired), reverting otherwise.
- Mirror the same invariant where unallocated withdrawals are validated in `PlasmaVault.withdraw` and `_redeem` so that all entry points respect pending request reservations.

Operational safeguards:

- Emit events when the definition or enforcement of unallocated balance changes, and surface metrics for `sharesToRelease`, `totalPendingRequestShares`, and available unallocated shares to aid monitoring.
- Add unit and invariant tests that demonstrate: unallocated withdrawals cannot reduce available liquidity below the amount needed to satisfy pending requests under the chosen semantics.

[Go back to Findings Summary](#)

W6: Hardcoded selector special-casing in `_checkCanCall` risks authorization drift and selector-collision misclassification

Impact:	Warning	Confidence:	Medium
Target:	contracts/managers/access/AccessManagedUpgradeable.sol	Type:	Access control

Description

`AccessManagedUpgradeable` overrides the authorization subject for two hardcoded function selectors by forcing `caller_ = msg.sender` instead of using `_msgSender`. This couples access-control semantics to magic 4-byte constants and makes the behavior fragile across upgrades and susceptible to accidental selector collisions.

If the signature of the context-management functions changes in a future release, the special-casing will silently stop applying because the selector will change. Conversely, if any unrelated function happens to share one of these 4-byte values, it will be treated as a context operation and authorized against `msg.sender` rather than `_msgSender`. Under meta-transaction or proxying schemes where `_msgSender` can differ from `msg.sender`, this can produce authorization drift, causing unintended allow/deny outcomes or a denial of service for context operations.

Listing 57. Code snippet from `contracts/managers/access/AccessManagedUpgradeable.sol`

```
130 function _checkCanCall(address caller_, bytes calldata data_) internal
    virtual {
131     bytes4 sig = bytes4(data_[0:4]);
132     if (sig == CONTEXT_MANAGER_SETUP || sig == CONTEXT_MANAGER_CLEAR) {
133         caller_ = msg.sender;
134     }
```

```

135
136     AccessManagedStorage storage $ = _getAccessManagedStorage();
137     (bool immediate, uint32 delay) =
138         AuthorityUtils.canCallWithDelay(authority(), caller_, address(this),
139         bytes4(data_[0:4]));
139     if (!immediate) {
140         if (delay > 0) {
141             $._consumingSchedule = true;
142             IAccessManager(authority()).consumeScheduledOp(caller_, data_);
143             $._consumingSchedule = false;
144         } else {
145             revert AccessManagedUnauthorized(caller_);
146         }
147     }
148 }

```

Wake function-selector resolution confirms the intention of the special-casing:

- 0x87ef0b87 resolves to `ContextClient.setupContext(address);` and
- 0xdb99bddd resolves to `ContextClient.clearContext();`

The corresponding functions are defined here.

*Listing 58. Code snippet from
contracts/managers/context/ContextClient.sol*

```

47 function setupContext(address sender_) external override restricted {
48     if (ContextClientStorageLib.isContextSenderSet()) {
49         revert ContextAlreadySet();
50     }
51
52     ContextClientStorageLib.setContextSender(sender_);
53
54     emit ContextSet(sender_);
55 }

```

*Listing 59. Code snippet from
contracts/managers/context/ContextClient.sol*

```

65 function clearContext() external override restricted {

```

```

66     address currentSender = ContextClientStorageLib.getSenderFromContext();
67
68     if (currentSender == address(0)) {
69         revert ContextNotSet();
70     }
71
72     ContextClientStorageLib.clearContextStorage();
73
74     emit ContextCleared(currentSender);
75 }

```

Impact in production deployments:

- In systems that later adopt ERC-2771 or other meta-transaction patterns where `_msgSender` differs from `msg.sender`, the hardcoded override changes who `AuthorityUtils.canCallWithDelay` evaluates. This can unintentionally authorize the trusted forwarder or deny valid end users for the affected selectors.
- Selector collisions are rare but possible. Any added function that collides with `0x87ef0b87` or `0xdb99bddd` would inherit the `msg.sender` override, altering its authorization semantics unexpectedly.

Exploit Scenario

An attacker can exploit the brittle selector special-casing when the system whitelists an infrastructure caller (for example, a trusted forwarder) and when a selector collision occurs or is introduced in an upgrade.

Consider the following attack scenario:

1. Bob deploys an upgrade that both enables meta-transactions and whitelists the trusted forwarder address in the `Authority` to call restricted functions.
2. The upgrade also introduces a new privileged function whose 4-byte selector (by rare coincidence) equals `0x87ef0b87`.

3. Alice routes a transaction for this function through the trusted forwarder so that `msg.sender` is the forwarder and `_msgSender` is Alice.
4. In `AccessManagedUpgradeable._checkCanCall`, the selector matches the hardcoded constant, so `caller_` is forced to `msg.sender` (the whitelisted forwarder).
5. `AuthorityUtils.canCallWithDelay` authorizes the call based on the forwarder being permitted, and the privileged function executes even though Alice herself lacks the required permission.

Production impact:

- Unauthorized execution of functions that accidentally collide with the special-cased selectors; or
- Denial of service for context operations after a signature change, because authorization is evaluated against a different subject than operators expect.

Recommendation

Prefer explicit, type-safe selector derivation and localize special-casing to the contracts that own the behavior.

Fix the vulnerability by either:

- Replace magic constants with interface-derived selectors, e.g., `bytes4 constant CONTEXT_MANAGER_SETUP = IContextClient.setupContext.selector` and `bytes4 constant CONTEXT_MANAGER_CLEAR = IContextClient.clearContext.selector`. This ensures the values track signature changes at compile time.
- Move the special-casing out of the base `AccessManagedUpgradeable` to a virtual hook. For example, add an internal `function _authorizationSubject(bytes4 selector, address defaultCaller) internal view virtual returns (address)` that returns `defaultCaller` by default. In

`ContextClient`, override it to return `msg.sender` only for `IContextClient.setupContext.selector` and `IContextClient.clearContext.selector`.

- Add unit tests that assert: (a) the special-cased selectors equal the interface selectors; and (b) no other function in the codebase shares those selectors. Fail CI if a collision is detected.

Operational safeguards:

- If adopting ERC-2771 or other meta-transaction schemes, document which selectors are evaluated against `msg.sender` and which against `_msgSender`, and ensure the `Authority` never broadly whitelists infrastructure addresses for privileged selectors.

[Go back to Findings Summary](#)

W7: Global redemption lock scoped per AccessManager (not per vault) enables cross-vault lockouts

Impact:	Warning	Confidence:	Medium
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Logic error

Description

The redemption lock is stored in the Access Manager's ERC-7201 storage and keyed only by `account`. The vault `target` is not part of the key. Consequently, when multiple vaults share the same `IporFusionAccessManager`, a deposit into any one vault sets a global lock for the receiver's `account`, and subsequent `withdraw`, `redeem`, `transfer`, or `transferFrom` operations initiated by that `account` against other vaults that use the same Access Manager will revert until the lock expires.

The following implementation details establish the cross-vault coupling:

- `RedemptionDelayLib.lockChecks` reads and writes `redemptionLock[account_]` and does not include any `target` dimension.

Listing 60. Code snippet from

contracts/managers/access/RedemptionDelayLib.sol

```
53 function lockChecks(address account_, bytes4 sig_) internal {
54     if (
55         sig_ == WITHDRAW_SELECTOR || sig_ == REDEEM_SELECTOR || sig_ ==
TRANSFER_FROM_SELECTOR
56         || sig_ == TRANSFER_SELECTOR
57     ) {
58         uint256 unlockTime =
IporFusionAccessManagersStorageLib.getRedemptionLocks().redemptionLock[account_];
59         if (unlockTime > block.timestamp) {
```



```

60         revert AccountIsLocked(unlockTime);
61     }
62     } else if (sig_ == DEPOSIT_SELECTOR || sig_ == MINT_SELECTOR || sig_ ==
        DEPOSIT_WITH_PERMIT_SELECTOR) {
63         IporFusionAccessManagersStorageLib.setRedemptionLocks(account_);
64     }
65 }

```

- The storage mapping is keyed by **address account** only, stored under the Access Manager's **REDEMPTION_LOCKS** slot.

Listing 61. Code snippet from

contracts/managers/access/IporFusionAccessManagersStorageLib.sol

(storage shape)

```

13 struct RedemptionLocks {
14     /// @notice Maps user addresses to their deposit timestamp
15     /// @dev Used to enforce redemption delays after deposits
16     mapping(address account => uint256 depositTime) redemptionLock;
17 }

```

- Locks are updated via **setRedemptionLocks(account_)**, again without the vault **target**.

Listing 62. Code snippet from

contracts/managers/access/IporFusionAccessManagersStorageLib.sol (write path)

```

112 function setRedemptionLocks(address account_) internal {
113     uint256 redemptionDelay = IIporFusionAccessManager(address(
        this)).REDEMPTION_DELAY_IN_SECONDS();
114     if (redemptionDelay == 0) {
115         return;
116     }
117     RedemptionLocks storage redemptionLocks = getRedemptionLocks();
118     uint256 redemptionLock = uint256(block.timestamp) + redemptionDelay;
119     redemptionLocks.redemptionLock[account_] = redemptionLock;
120     emit RedemptionDelayForAccountUpdated(account_, redemptionLock);
121 }

```

- `IporFusionAccessManager.canCallAndUpdate` calls `lockChecks(caller_, selector_)` and ignores `target_`, making the lock global per Access Manager instance.

Listing 63. Code snippet from `contracts/managers/access/IporFusionAccessManager.sol`

```
163 function canCallAndUpdate(address caller_, address target_, bytes4
    selector_)
164     external
165     override
166     returns (bool immediate, uint32 delay)
167 {
168     RedemptionDelayLib.lockChecks(caller_, selector_);
169     return super.canCall(caller_, target_, selector_);
170 }
```

- `PlasmaVault._checkCanCall` wires deposits to set the lock for the deposit `receiver`, and wires withdrawals, redemptions, and transfers to enforce the lock for the `owner/caller` via the same Access Manager. When several vaults share one Access Manager, a deposit in Vault A locks the `account` for Vault B as well.

Listing 64. Code snippet from `contracts/vaults/PlasmaVault.sol`

```
1390 function _checkCanCall(address caller_, bytes calldata data_) internal
    virtual override {
1391     bytes4 sig = bytes4(data_[0:4]);
1392     bool immediate;
1393     uint32 delay;
1394
1395     if (this.transferFrom.selector == sig) {
1396         (address tranferFromAddress,,) = abi.decode(_msgData()[4:],
            (address, address, uint256));
1397
1398         IporFusionAccessManager(authority()).canCallAndUpdate(tranferFromAddress,
            address(this), sig);
1399         (immediate, delay) =
            IporFusionAccessManager(authority()).canCallAndUpdate(caller_, address(
                this), sig);
1399     } else if (this.deposit.selector == sig || this.mint.selector == sig) {
```

```

1400         (, address receiver) = abi.decode(_msgData()[4:], (uint256,
address));
1401         IporFusionAccessManager(authority()).canCallAndUpdate(receiver,
address(this), sig);
1402         (immediate, delay) = AuthorityUtils.canCallWithDelay(authority(),
caller_, address(this), sig);
1403     } else if (this.depositWithPermit.selector == sig) {
1404         (, address receiver,,,,) = abi.decode(_msgData()[4:], (uint256,
address, uint256, uint8, bytes32, bytes32));
1405         IporFusionAccessManager(authority()).canCallAndUpdate(receiver,
address(this), sig);
1406         (immediate, delay) = AuthorityUtils.canCallWithDelay(authority(),
caller_, address(this), sig);
1407     } else if (this.redeem.selector == sig || this.withdraw.selector ==
sig) {
1408         (, address owner) = abi.decode(_msgData()[4:], (uint256, address,
address));
1409         IporFusionAccessManager(authority()).canCallAndUpdate(owner,
address(this), sig);
1410         (immediate, delay) =
IporFusionAccessManager(authority()).canCallAndUpdate(caller_, address(
this), sig);
1411     } else if (this.transfer.selector == sig) {
1412         (immediate, delay) =
IporFusionAccessManager(authority()).canCallAndUpdate(caller_, address(
this), sig);
1413     } else {
1414         (immediate, delay) = AuthorityUtils.canCallWithDelay(authority(),
caller_, address(this), sig);
1415     }
1416 }

```

Affected surface:

- Functions: `RedemptionDelayLib.lockChecks`,
`IporFusionAccessManager.canCallAndUpdate`, `PlasmaVault._checkCanCall`.
- State: `IporFusionAccessManagersStorageLib.RedemptionLocks.redeemptionLock` keyed only by `account` in the Access Manager's storage.

Realistic impact:

- If multiple vaults (e.g., Vault A and Vault B) share one

`IporFusionAccessManager`, a deposit into Vault A with `receiver = X` sets X's lock globally. Any `withdraw`, `redeem`, `transfer`, or `transferFrom` by X in Vault B will revert with `AccountIsLocked` until `unlockTime`.

- This creates a cross-vault denial-of-service vector and operational coupling between otherwise independent vaults. A minimal “dust” deposit is sufficient to trigger the lock.

Exploit Scenario

An attacker can force a cross-vault lockout for a victim when different vaults share the same Access Manager.

Consider the following attack scenario:

1. Alice funds a small amount of the deposit asset in Vault A (which uses the same `IporFusionAccessManager` as Vault B);
2. Alice calls `deposit(1, receiver = Bob)` on Vault A, setting Bob's global redemption lock in the shared Access Manager;
3. Bob later calls `withdraw` on Vault B to exit his position; and
4. Vault B routes Bob's call through `canCallAndUpdate`, which sees Bob's lock and reverts with `AccountIsLocked` until the lock expires. Alice can repeat dust deposits to keep Bob perpetually locked across all vaults sharing the Access Manager.

Recommendation

Scope redemption locks per vault instead of per Access Manager to eliminate cross-vault coupling.

Fix the vulnerability by either:

- Keying locks by `(vault target, account)`:
- In `IporFusionAccessManagersStorageLib`, change storage to `mapping(address`

`vault => mapping(address account => uint256)) redemptionLock;` under the same ERC-7201 slot;

- Update `setRedemptionLocks` to `setRedemptionLocks(address vault_, address account_)` and write to `redemptionLock[vault_][account_]`;
- Update `RedemptionDelayLib.lockChecks` to accept `account_`, `target_`, and `sig_`, and to read/write `redemptionLock[target_][account_]` for deposit/withdraw flows;
- In `IporFusionAccessManager.canCallAndUpdate`, pass `target_` into `lockChecks` so the correct vault dimension is used; and
- Update any reader, e.g., `getAccountLockTime`, to expose per-vault lock times or add a new `getAccountLockTime(address vault_, address account_)`.
- Or, enforce one Access Manager per vault so that the existing account-only key does not span multiple vaults. Document and test this invariant rigorously if chosen.

Additional hardening (optional):

- If business requirements permit, only set a redemption lock when `receiver == caller_` for deposits/mints, or add an explicit allowlist/consent mechanism for third-party deposits. This reduces griefing vectors but is secondary to proper per-vault scoping.

Validation steps after the fix:

- Unit-test that a deposit into Vault A does not affect the ability to withdraw from Vault B when both are governed by the same code;
- Assert that `getAccountLockTime` for `(vaultA, Bob)` differs from `(vaultB, Bob)` as expected; and
- Verify migration/initialization preserves zero locks for untouched vault/account pairs.

[Go back to Findings Summary](#)

I1: Y2106 timestamp downcast in fee timestamp update can revert and halt management fee realization (triggered by `prepareForRealizeManagementFee`)

Impact:	Info	Confidence:	High
Target:	<code>./contracts/vaults/lib/PlasmaVaultFeesLib.sol</code>	Type:	Arithmetics

Description

This library uses `updateManagementFeeData` to persist the last fee accrual timestamp. That function stores `block.timestamp` into `ManagementFeeData.lastUpdateTimestamp` via `SafeCast.toUint32`.

Once the Unix timestamp exceeds $2^{32} - 1$ (4294967295, approximately 2106-02-07T06:28:15Z), `SafeCast.toUint32` reverts. Any flow that calls `updateManagementFeeData` will then hard-revert, preventing management fee realization and potentially interrupting deposit/withdraw paths that rely on timestamp updates.

The call is reachable from `PlasmaVaultFeesLib.prepareForRealizeManagementFee`, which calculates the unrealized management fee and then updates the timestamp.

Affected elements:

- State variable:
`PlasmaVaultStorageLib.ManagementFeeData.lastUpdateTimestamp (uint32)`.
- Functions: `PlasmaVaultFeesLib.prepareForRealizeManagementFee` and `PlasmaVaultLib.updateManagementFeeData`.

*Listing 65. Code snippet from
contracts/vaults/lib/PlasmaVaultFeesLib.sol*

```
54 recipient = feeData.feeAccount;
55
56 unrealizedFeeInUnderlying = getUnrealizedManagementFee(totalAssetsBefore_);
57
58 PlasmaVaultLib.updateManagementFeeData();
59 }
```

The timestamp write that may revert after the Year-2106 boundary is shown below.

Listing 66. Code snippet from contracts/libraries/PlasmaVaultLib.sol

```
447 function updateManagementFeeData() internal {
448     PlasmaVaultStorageLib.ManagementFeeData storage feeData =
        PlasmaVaultStorageLib.getManagementFeeData();
449     feeData.lastUpdateTimestamp = block.timestamp.toUint32();
450 }
```

The root cause is the `uint32` storage type used for the timestamp.

*Listing 67. Code snippet from
contracts/libraries/PlasmaVaultStorageLib.sol*

```
962 struct ManagementFeeData {
963     address feeAccount;
964     uint16 feeInPercentage;
965     uint32 lastUpdateTimestamp;
966 }
```

Impact assessment:

- Before 2106-02-07, behavior is correct. After that date, the cast reverts, making `prepareForRealizeManagementFee` and any other path that touches `updateManagementFeeData` fail.
- Realistic production risk is long-term but concrete: long-lived

deployments would face a time-triggered, permanent DoS of fee realization, and any flows that update the fee timestamp could also be affected.

Exploit Scenario

The issue is time-triggered and does not require a privileged attacker; however, any user can surface the failure by invoking a function that reaches `updateManagementFeeData` after the Year-2106 boundary.

Consider the following attack scenario:

1. Alice calls a function that realizes or prepares to realize management fees after the Unix timestamp has exceeded $2^{32} - 1$;
2. The call flows into `prepareForRealizeManagementFee`, which invokes `updateManagementFeeData` to persist the current time;
3. `SafeCast.toUint32` attempts to downcast `block.timestamp` and reverts; and
4. Bob, the fee recipient and other users, observe that fee realization (and potentially dependent deposit/withdraw paths) no longer works.

Recommendation

Use a timestamp storage type with sufficient headroom and remove the lossy downcast.

Recommended changes:

- Change `ManagementFeeData.lastUpdateTimestamp` from `uint32` to a wider type such as `uint48` or `uint64` (both provide centuries of headroom);
- Replace `block.timestamp.toUint32()` with `block.timestamp.toUint48()` (or `toUint64()`), matching the chosen storage type; and
- Review all arithmetic that reads `lastUpdateTimestamp` to ensure implicit widening remains safe (it already uses 256-bit math in fee calculations).

If changing storage layout is not immediately feasible in existing deployments, harden the write to avoid a revert after 2106 as a stopgap:

- Guard the cast and saturate at the maximum representable value to avoid bricking the system:
- `feeData.lastUpdateTimestamp = uint32(block.timestamp > type(uint32).max ? type(uint32).max : block.timestamp);`
- Note: this prevents reverts but freezes accrual progression; plan a migration to a wider type.

Add tests that assert Year-2106 behavior and document the storage width assumption in code comments to prevent regressions.

[Go back to Findings Summary](#)

I2: Negative price answers revert via SafeCast before domain-specific checks, causing inconsistent errors

Impact:	Info	Confidence:	High
Target:	./contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol	Type:	Code quality

Description

The conversion from an oracle's signed `answer` to unsigned `assetPrice` is done via `SafeCast.toUint256` before the contract's semantic validation. If a feed returns a negative value, the call reverts with a generic `SafeCast` error rather than the middleware's domain-specific error, producing inconsistent error semantics and complicating monitoring.

While negative prices are unlikely for USD feeds, custom `IPriceFeed` implementations can return negative sentinel values or bugs. The current order prevents the contract from surfacing its intended `UnexpectedPriceResult` error.

Affected elements:

- Function: `_getAssetPrice`
- Local variables: `priceFeedPrice`, `priceFeedDecimals`

Listing 68. Code snippet from

contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol

```
232         assetPrice = IporMath.convertToWad(priceFeedPrice.toUint256(),
        priceFeedDecimals);
233
234         if (assetPrice <= 0) {
235             revert IPriceOracleMiddleware.UnexpectedPriceResult();
```

A negative `priceFeedPrice` will revert at `toUint256()` and never reach the domain-specific check below.

Exploit Scenario

An attacker who controls a custom price feed can force a generic `SafeCast` revert instead of the contract's intended domain-specific error, reducing observability and making alerts less actionable.

Consider the following attack scenario:

- Bob configures a custom `IPriceFeed` as the source for an asset;
- Alice operates that feed and intentionally returns a negative `answer` for one round;
- The middleware attempts to cast the negative value to `uint256` and reverts with a generic `SafeCast` error string instead of `UnexpectedPriceResult`;
- Bob's monitoring, which keys off domain-specific errors, fails to trigger, delaying detection and incident response.

Realistic impact:

- Denial of service for consumers and degraded monitoring fidelity due to inconsistent revert reasons.

Recommendation

Validate the signed `answer` before any casts and revert using a domain-specific error for all invalid values.

Suggested changes inside `_getAssetPrice`:

- Check `priceFeedPrice <= 0` and revert with `UnexpectedPriceResult` (or a dedicated error) before calling `toUint256()`;
- Only after passing this check, convert to WAD and proceed with further logic.

This preserves consistent error semantics across both custom feeds and Chainlink fallback.

[Go back to Findings Summary](#)

I3: Misleading NatSpec: function documented as public/no restrictions but guarded by AccessManager **restricted** modifier

Impact:	Info	Confidence:	High
Target:	contracts/vaults/PlasmaVault.sol	Type:	Code quality

Description

The `updateMarketsBalances` external function is decorated with the **restricted** modifier from `AccessManagedUpgradeable`, which enforces authorization through the `AccessManager` based on the caller and the function selector.

However, its NatSpec declares `@custom:access Public function, no role restrictions`, which is incorrect. This mismatch can mislead integrators, off-chain automation, and security tooling into assuming open access where the call path is, in reality, restricted. Such inconsistencies create operational risks (failed transactions, stuck automation) and complicate audits and maintenance.

The evidence below shows the NatSpec stating no restrictions while the function is guarded by **restricted**.

Listing 69. Code snippet from contracts/vaults/PlasmaVault.sol

```
380 ----    /// @param marketIdspass:[] Array of market IDs to update
381    /// @return uint256 Updated total assets after balance refresh
382    /// @custom:access Public function, no role restrictions
383    function updateMarketsBalances(uint256[] calldata marketIdspass:[])
        external restricted returns (uint256) {
384        if (marketIdspass:[].length == 0) {
385            return totalAssets();
386        }
387        uint256 totalAssetsBefore = totalAssets();
388        pass:[]updateMarketsBalances(marketIdspass:[]);
```

```
389         pass:[_]addPerformanceFee(totalAssetsBefore);
```

```
        return totalAssets();  
    }
```

The `restricted` modifier implementation demonstrates that access is checked against the configured `authority` via `pass:[_]checkCanCall`, which looks up permissions for the current caller and function selector. Therefore, the function is not publicly callable without an appropriate permission in the `AccessManager`.

.Code snippet from `contracts/managers/access/AccessManagedUpgradeable.sol`
[source,solidity,linenums,start=87]

 modifier restricted() {
 pass:[_]checkCanCall(pass:[_]msgSender(), pass:[_]msgData());
 pass:[_];
 }

Exploit Scenario

Although this is not a direct vulnerability in access control (the code is stricter than the documentation), it creates a realistic operational failure mode when integrators rely on the documentation.

Consider the following scenario:

1. Alice integrates the vault and builds an off-chain keeper that calls `updateMarketsBalances`, relying on the NatSpec that states no role restrictions.
2. Alice deploys the automation in production without provisioning the required permission in `AccessManager`.
3. The keeper transactions revert due to the `restricted` modifier; balances and fee calculations are not refreshed as expected.
4. Bob, operating a downstream analytics or accounting pipeline, observes stale totals and reports incorrect KPIs, leading to operational incidents

and potential financial decisions based on outdated data.

Impact in production can include failed jobs, stale accounting, delayed fee realization, and misconfigured permissions that require emergency changes in the access manager.

Recommendation

Align the documentation with the implemented access control and make the contract's expectations explicit.

Recommended actions:

- Update the NatSpec `@custom:access` for `updateMarketsBalances` to reflect that the function is `external` and guarded by the `restricted` modifier, specifying the role or policy that is expected to be authorized in `AccessManager` (for example, an automation/keeper role or scheduled operation).
- If the intention was to allow open access, remove the `restricted` modifier and document the rationale; otherwise, keep `restricted` and correct the docs.
- Add unit tests that assert unauthorized EOAs revert with `AccessManagedUnauthorized` while authorized callers succeed, ensuring documentation stays in sync with behavior.
- Optionally add a repository-wide guideline stating that `@custom:access` must always match effective modifiers and the access policy enforced by `AccessManager`.

[Go back to Findings Summary](#)

14: Misleading access control: NatSpec says owner-only, but function is role-gated (SET_ASSETS_PRICES_SOURCES)

Impact:	Info	Confidence:	High
Target:	contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol	Type:	Code quality

Description

The function `setAssetsPricesSources` is documented as "Only callable by owner", but the implementation uses OpenZeppelin `AccessControl` and gates the function with modifier `onlyRole` using role `SET_ASSETS_PRICES_SOURCES`. The contract does not implement `Ownable` and therefore has no `owner` concept. This discrepancy between documentation and code can cause operators and integrators to misunderstand who is authorized to call the function and how permissions must be configured.

Affected components:

- Function `setAssetsPricesSources` in `contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol`.
- Role constant `SET_ASSETS_PRICES_SOURCES` defined in `contracts/price_oracle/IporFusionAccessControl.sol`.

The NatSpec immediately above the function claims owner-only access, while the code enforces role-based access control. Evidence below shows the mismatch.

Listing 70. Code snippet from

contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol

```
98 /// @notice Sets or updates price feed sources for multiple assets
```

```

99 /// @param assets_ Array of asset addresses
100 /// @param sources_ Array of corresponding price feed sources
101 /// @dev Arrays must be equal length and non-empty
102 /// @dev Only callable by owner
103 function setAssetsPricesSources(address[] calldata assets_, address[]
    calldata sources_)
104     external
105     onlyRole(SET_ASSETS_PRICES_SOURCES)
106 {

```

The role used by the modifier is declared in the access control base contract.

*Listing 71. Code snippet from
contracts/price_oracle/IporFusionAccessControl.sol*

```

16 bytes32 public constant ADD_PT_TOKEN_PRICE = keccak256("ADD_PT_TOKEN_PRICE");
17
18 /// @notice Role that allows setting assets prices sources
19 /// @dev bytes32 -
    0x58fb5220de46b94ead43a7c850443ec6b00bec9e9e2a8741abda98af086ec957
20 bytes32 public constant SET_ASSETS_PRICES_SOURCES = keccak256
    ("SET_ASSETS_PRICES_SOURCES");

```

Realistic impact in production:

- Teams may assume an **owner** (e.g., a multisig) can execute **setAssetsPricesSources**, but calls will revert unless the caller has **SET_ASSETS_PRICES_SOURCES**. In emergencies this can delay price source changes and keep feeds stale.
- Conversely, teams might grant **SET_ASSETS_PRICES_SOURCES** to a broader set of operators while believing only an **owner** can actually execute changes. This increases the blast radius and could allow unintended parties to repoint price sources.

Exploit Scenario

An operator can leverage the documentation mismatch to gain or retain broader permissions than intended when roles are configured based on the

misleading NatSpec.

Consider the following attack scenario:

1. Alice is a semi-trusted operations engineer. The team relies on the NatSpec that says "Only callable by owner" and assumes only their multisig can change price sources.
2. Alice is mistakenly granted the role `SET_ASSETS_PRICES_SOURCES` for "monitoring" duties because the team believes the role alone is harmless.
3. Alice calls `setAssetsPricesSources` and sets the source of a tracked asset to a feed she controls.
4. The contract accepts the change because access is enforced by modifier `onlyRole` with role `SET_ASSETS_PRICES_SOURCES`, not by ownership.
5. Bob's protocol integrates this oracle and begins consuming Alice's manipulated price, leading to incorrect valuations and potential loss.

Recommendation

Align the specification and the implementation so that authorized callers are unambiguous and consistently enforced.

Fix the discrepancy by either:

- Keeping role-based access control and updating NatSpec to: "Only callable by addresses that have the role `SET_ASSETS_PRICES_SOURCES``". Also document how to grant and revoke this role (e.g., ``grantRole(SET_ASSETS_PRICES_SOURCES, <multisig>`)`) and who administers it.
- If the business requirement is truly owner-only access, enforce it in code: inherit `OwnableUpgradeable` and guard the function with `onlyOwner`, or alternatively restrict it to `onlyRole(DEFAULT_ADMIN_ROLE)` while ensuring that only a single admin (the intended owner) holds that role. Update

NatSpec accordingly.

Add a test or lint check to validate that NatSpec authorization statements match the actual modifier on each function.

[Go back to Findings Summary](#)

I5: Unused struct CallbackHandlerInfo increases code surface without benefit

Impact:	Info	Confidence:	High
Target:	contracts/readers/CallbackHandlerReader.sol	Type:	Unused code

Description

The `CallbackHandlerInfo` struct is defined but never used anywhere in the repository. Keeping unused types increases code surface and maintenance overhead, makes audits noisier, and may mislead integrators into expecting functionality that does not exist.

*Listing 72. Code snippet from
contracts/readers/CallbackHandlerReader.sol*

```
12 struct CallbackHandlerInfo {
13     /// @notice Address of the protocol contract that triggers callbacks
14     address sender;
15     /// @notice Function signature that identifies the callback
16     bytes4 sig;
17     /// @notice Address of the callback handler implementation contract
18     address handler;
19 }
```

Realistic impact:

- Developers and auditors may infer planned or existing features around returning this struct, leading to confusion or incorrect integrations.
- Dead code marginally increases bytecode size and surface area for mistakes in future refactors.

Exploit Scenario

Consider the following scenario:

1. Alice reads the code and assumes there is or will be an interface that returns `CallbackHandlerInfo` as part of the public API.
2. She builds off-chain tooling that depends on this struct being returned by the reader.
3. Later, when interacting with Bob's protocol, the tooling fails or misinterprets results because the struct is never actually used or returned.
4. Bob's operations are delayed while the integration is fixed, as the unused type misled the implementation.

Recommendation

Remove unused code to reduce confusion and maintenance burden:

- Delete the `CallbackHandlerInfo` struct from `CallbackHandlerReader.sol` if it is not used anywhere.
- If the struct is intended for future use, move it next to the code that uses it and add NatSpec documenting where and how it is consumed. Alternatively, place it in a dedicated types file that is actually imported by consumers.
- As a general practice, enable CI checks (for example, `solhint` or static analysis) to flag unused types and identifiers.

[Go back to Findings Summary](#)

I6: NatSpec promises decimals-mismatch reverts, but implementation never validates decimals

Impact:	Info	Confidence:	High
Target:	contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol	Type:	Code quality

Description

The NatSpec for `_getAssetPrice` states it will revert if a feed uses decimals different from `QUOTE_CURRENCY_DECIMALS`. The code does not enforce this: it always reads the feed-reported decimals, converts using `IporMath.convertToWad`, and returns `decimals = QUOTE_CURRENCY_DECIMALS` without any equality check.

This discrepancy can mislead integrators and auditors into relying on behavior that does not exist, complicating incident response and monitoring when a misconfigured feed is attached.

Affected elements:

- Function: `_getAssetPrice`
- State constants: `QUOTE_CURRENCY_DECIMALS`

Listing 73. Code snippet from

contracts/price_oracle/PriceOracleMiddlewareWithRoles.sol

```
192    /// @dev Tries custom price feed first, falls back to Chainlink Registry
    if no custom feed is set
193    /// @dev Reverts if:
194    /// @dev - asset_ is zero address
195    /// @dev - price <= 0
196    /// @dev - decimals don't match QUOTE_CURRENCY_DECIMALS
197    /// @dev - no custom price feed is set and CHAINLINK_FEED_REGISTRY is
    address(0)
198    /// @dev - asset is not supported in Chainlink Registry when using it as
```

```
fallback
199      /// @dev - Chainlink Registry call fails
```

In reality the implementation converts using the feed's own decimals and never throws a decimals-mismatch error.

Exploit Scenario

The mismatch is informational but can have operational consequences if teams expect a revert on decimals mismatch and build automation around that expectation.

Consider the following attack scenario:

- Bob assumes `_getAssetPrice` will revert on any decimals mismatch as the NatSpec claims and configures on-chain guards accordingly;
- Alice deploys a custom feed for an asset with non-standard decimals and sets it as the source (via authorized role);
- The middleware accepts the feed and silently converts values without reverting, bypassing Bob's guardrails that relied on the promised revert behavior;
- Bob's monitoring and off-chain tooling miss the configuration error because no revert occurs, increasing the time-to-detect and amplifying potential mispricing risk.

Realistic impact:

- Misleading documentation undermines defense-in-depth and monitoring assumptions.

Recommendation

Make the implementation match the documentation or update the documentation to match reality.

Two viable options:

- Enforce the documented constraint: after obtaining `priceFeedDecimals`, revert with a domain-specific error if `priceFeedDecimals != QUOTE_CURRENCY_DECIMALS`;
- If the current design (automatic conversion to WAD regardless of feed decimals) is desired, remove the misleading NatSpec lines and clearly document that the middleware normalizes decimals rather than validating equality.

Prefer explicit, test-covered behavior and add unit tests asserting the chosen semantics.

[Go back to Findings Summary](#)

Appendix A: How to cite

Please cite this document as:

[Ackee Blockchain Security](#), Wake Arena Report | IPOR-Labs: ipor-fusion,
December 9, 2025 13:19 UTC.



Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz

